

ИНСТИТУТ ИНФОРМАЦИОННЫХ ТЕХНОЛОГИЙ И КОМПЬЮТЕРНЫХ НАУК  
КАФЕДРА АВТОМАТИЗИРОВАННЫХ СИСТЕМ УПРАВЛЕНИЯ  
НАПРАВЛЕНИЕ 09.03.01 ИНФОРМАТИКА И ВЫЧИСЛИТЕЛЬНАЯ ТЕХНИКА

---

# ВЫПУСКНАЯ КВАЛИФИКАЦИОННАЯ РАБОТА БАКАЛАВРА

на тему: «Оценка стоимости ремонтных работ в автосервисе с применением методов  
машинного обучения»

---

|                                     |                               |
|-------------------------------------|-------------------------------|
| Студент                             | Бутаков Богдан Игоревич       |
| Руководитель работы                 | Агабубаев Аслан Такабудинович |
| Нормоконтроль проведен              | Агабубаев А.Т.                |
| Проверка на заимствования проведена | Овчинников С.А.               |

Работа рассмотрена кафедрой и допущена к защите в ГЭК

---

|                     |              |
|---------------------|--------------|
| Заведующий кафедрой | Темкин И.О.  |
| Директор института  | Солодов С.В. |

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ  
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ  
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ ТЕХНОЛОГИЧЕСКИЙ  
УНИВЕРСИТЕТ «МИСИС»

---

УТВЕРЖДАЮ

Институт ИТКН

Кафедра АСУ

Направление 09.03.01 ИВТ

Зав. кафедрой

Темкин И.О.

«18» декабря 2025 г.

**ЗАДАНИЕ  
НА ВЫПОЛНЕНИЕ ВЫПУСКНОЙ  
КВАЛИФИКАЦИОННОЙ РАБОТЫ БАКАЛАВРА**

Студенту группы БИВТ-22-16, Бутакову Богдану Игоревичу

(номер группы, Ф.И.О. полностью)

1. Тема работы Оценка стоимости ремонтных работ в автосервисе с применением методов машинного обучения

2. Цель работы Повышение эффективности процесса оценки стоимости ремонта поврежденных автомобилей

3. Исходные данные Самостоятельно собранные и размеченные фотографии повреждённых автомобилей (реальные обращения клиентов, публичные и научные датасеты)

4. Основная литература, в том числе:

4.1. Монография, учебники и т. п. Не предусмотрены

4.2. Отчёты по НИР, диссертации, дипломные работы и т. п. Не предусмотрены

4.3. Периодическая литература

Automated Car Damage Assessment Using Computer Vision: Insurance Company Use Case. — 2024. — URL: <https://www.mdpi.com/2076-3417/14/20/9560> ; [Online]. Applied Sciences (MDPI), Vol. 14, No. 20

HL-YOLO: Improving Vehicle Damage Detection with Heterogeneous Convolutions and Large-Kernel Attention. — 2025. — URL: <https://www.mdpi.com/2032-6653/16/12/640> ; [Online]. World Electric Vehicle Journal (MDPI), Vol. 16, No. 12.

Evaluation of deep learning algorithms for semantic segmentation of carparts. — 2021. — URL: [https://www.researchgate.net/publication/351787893\\_Evaluation\\_of\\_deep\\_learning\\_algorithms\\_for\\_semantic\\_segmentation\\_of](https://www.researchgate.net/publication/351787893_Evaluation_of_deep_learning_algorithms_for_semantic_segmentation_of) ;[Online]. ResearchGate / preprint.

4.4. Справочники и методическая литература (в том числе литература по методам обработки экспериментальных данных)

CVAT: Computer Vision Annotation Tool — Documentation. — 2024. — URL: <https://docs.cvat.ai/docs/> ; [Online]. Online documentation.

5. Перечень основных этапов исследования и форма промежуточной отчётности по каждому этапу

1. Анализ научно-технических источников, выбор подходов/методов для реализации — письменный отчёт
2. Проектирование архитектуры решения — письменный отчёт и архитектурные диаграммы
3. Формирование размеченного датасета, разработка и обучение ML-модели — письменный отчёт и программная реализация
4. Разработка сервисов, интеграция ML части в общий функционал, тестирование и развёртывание системы — письменный отчёт, программная реализация

6. Аппаратура и методики, которые должны быть использованы в работе Специализированного оборудования не применяется

7. Использование ЭВМ

1. AMD Ryzen 5 5600H with Radeon Graphics (6C/12T), RAM 16 ГБ, SSD, x86\_64

8. Перечень (примерный) основных вопросов, которые должны быть рассмотрены и проанализированы в литературном обзоре

1. Архитектуры моделей для анализа изображений
2. Сравнительная применимость моделей к задаче обнаружения деталей и повреждений
3. Исследование подходов, направленных на повышение качества работы модели

9. Перечень (примерный) графического и иллюстрированного материала

1. Схемы, иллюстрирующие принципы примененных методов машинного обучения
2. BPMN-диаграммы бизнес-процесса в состояниях as is и to be
3. Диаграммы, иллюстрирующие архитектуру проектируемого сервиса
4. Схема таблиц базы данных
5. Иллюстрации результатов работы модели

10. Руководитель работы Старший преподаватель кафедры АСУ, Агабубаев Аслан Такабудинович  
(должность, уч. степень, звание, Ф.И.О.)

(подпись)

11. Консультанты по работе (с указанием относящихся к ним разделов) Консультантов по работе нет

Дата выдачи задания \_\_\_\_\_ «18» декабря 2025 г.

**Задание принял к исполнению студент** \_\_\_\_\_

(подпись)

## **АННОТАЦИЯ**

Выпускная квалификационная работа изложена на 74 страницах, содержит 37 рисунков, 10 таблиц, 22 источника, 1 приложение.

### **ОБНАРУЖЕНИЕ ПОВРЕЖДЕНИЙ АВТОМОБИЛЕЙ, СВЁРТОЧНЫЕ НЕЙРОННЫЕ СЕТИ, ОБРАБОТКА ИЗОБРАЖЕНИЙ, КОМПЬЮТЕРНОЕ ЗРЕНИЕ, ГЛУБОКОЕ ОБУЧЕНИЕ**

Данная выпускная квалификационная работа посвящена автоматизации процесса предварительной оценки повреждений автомобилей по фотографиям. Объектом исследования в данной работе является процесс оценки стоимости ремонта автомобилей в автосервисе. Целью работы является повышение эффективности процесса оценки стоимости ремонта поврежденных автомобилей, а её актуальность обусловлена ростом количества фотообращений клиентов и развитием методов компьютерного зрения и глубокого обучения. В рамках работы исследованы и применены современные методы глубокого обучения и компьютерного зрения. Для решения задачи обучены модели YOLOv8-seg на самостоятельно собранном и размеченном датасете изображений повреждённых автомобилей. Реализована система, основанная на микросервисной архитектуре с бизнес-логикой расчёта стоимости и длительности работ и интеграцией с ботом в мессенджере. В результате работы получен сервис автоматизированной оценки повреждений, позволивший ускорить обработку обращений на 29 минут.



## **ABSTRACT**

The Bachelor's thesis has 74 pages, 37 figures, 10 tables, 22 references, 1 appendix.

**AUTO DAMAGE DETECTION, CONVOLUTIONAL NEURAL NETWORK,  
IMAGE PROCESSING, COMPUTER VISION, DEEP LEARNING**

This final qualifying work is devoted to automating the process of preliminary assessment of car damage based on photographs. The object of this research is the process of estimating the cost of car repairs in a car service. The aim of the work is to increase the efficiency of the process of estimating the repair cost of damaged vehicles, and its relevance is due to the growing number of photo requests from customers and the development of computer vision and deep learning methods. As part of the work, modern methods of deep learning and computer vision have been studied and applied. To solve the problem, the YOLOv8-seg models were trained on a self-assembled and labeled dataset of images of damaged cars. A system based on a microservice architecture with business logic for calculating the cost and duration of work and integration with a messenger bot has been implemented. As a result, an automated damage assessment service was obtained, which made it possible to speed up the processing of requests by 29 minutes.

## СОДЕРЖАНИЕ

|                                                                                         |    |
|-----------------------------------------------------------------------------------------|----|
| ВВЕДЕНИЕ . . . . .                                                                      | 8  |
| 1 Обзор научно-технических источников информации . . . . .                              | 10 |
| 1.1 Свёрточные нейронные сети в анализе изображений . . . . .                           | 11 |
| 1.2 Разделение задачи на подзадачи: детали, повреждения и<br>оценка стоимости . . . . . | 13 |
| 1.3 Подходы к распознаванию деталей автомобиля . . . . .                                | 14 |
| 1.3.1 Mask R-CNN . . . . .                                                              | 15 |
| 1.3.2 Global Context Network (GCNet) . . . . .                                          | 15 |
| 1.3.3 Hybrid Task Cascade (HTC) . . . . .                                               | 16 |
| 1.3.4 Усовершенствованная архитектура на базе YOLO . . . . .                            | 17 |
| 1.4 Подходы к распознаванию повреждений автомобиля . . . . .                            | 18 |
| 2 Структурный системный анализ исследуемого объекта . . . . .                           | 23 |
| 3 Постановка задачи . . . . .                                                           | 26 |
| 4 Архитектурно-техническое решение задачи . . . . .                                     | 28 |
| 5 Математическая модель решения задачи . . . . .                                        | 33 |
| 5.1 Архитектура решения в нотации C4 . . . . .                                          | 33 |
| 5.2 Схема базы данных . . . . .                                                         | 37 |
| 5.3 Архитектура и математическая модель ML-составляющей . . . . .                       | 38 |
| 5.3.1 Модель детекции и сегментации деталей автомобиля . . . . .                        | 39 |
| 5.3.2 Формирование обрезанных областей деталей автомобиля . . . . .                     | 40 |
| 5.3.3 Модель детекции и сегментации повреждений . . . . .                               | 40 |
| 5.3.4 Формирование финального аннотированного изображения . . . . .                     | 42 |
| 5.4 Математическая модель расчёта стоимости и длительности<br>ремонтных работ . . . . . | 42 |
| 6 Алгоритм решения задачи . . . . .                                                     | 44 |
| 7 Программная реализация . . . . .                                                      | 47 |

|                                                                  |                                                                            |    |
|------------------------------------------------------------------|----------------------------------------------------------------------------|----|
| 7.1                                                              | Программная реализация пайплайна обучения моделей и<br>инференса . . . . . | 47 |
| 7.2                                                              | Программная реализация серверной части . . . . .                           | 48 |
| 7.3                                                              | Реализация интерфейса в мессенджере . . . . .                              | 49 |
| 7.4                                                              | Тестирование . . . . .                                                     | 50 |
| 8                                                                | Анализ результатов . . . . .                                               | 52 |
| 8.1                                                              | Результаты обучения моделей компьютерного зрения . . . .                   | 52 |
| 8.2                                                              | Результаты нагрузочного тестирования . . . . .                             | 59 |
| 8.3                                                              | Результаты работы сервиса . . . . .                                        | 60 |
| 8.4                                                              | Оценка выполнения задач исследования . . . . .                             | 67 |
| ЗАКЛЮЧЕНИЕ . . . . .                                             |                                                                            | 69 |
| СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ . . . . .                       |                                                                            | 71 |
| Приложение А Исходный код и описание структуры репозитория . . . |                                                                            | 74 |

## ВВЕДЕНИЕ

Автомобиль сегодня является основным средством передвижения для большей части населения, а ДТП и последующий ремонт - нередкое явление. При повреждении автомобиля, перед тем, как отдать его в ремонт, владельцы хотят иметь представление, сколько потребуется платить за услуги и ожидать их. Однако получить такую информацию быстро и не приезжая в автосервис выходит далеко не всегда.

Сотрудники автосервисов ежедневно вынуждены выделять время на обработку такого рода обращений по оценке повреждений от потенциальных клиентов. Если клиент отправляет фотографию мастеру, он вынужден ждать, пока сотрудник выйдет на связь и даст ответ. Если же требуется приехать в автосервис лично, клиент вынужден тратить дополнительное время на дорогу, оставаясь без понимания стоимости и длительности ремонта. Это потенциально может вызывать дискомфорт и снижать вероятность обращения, если, например, в другом автосервисе процессы устроены более прозрачно. В обоих случаях персоналу нужно отвлекаться от работы, чтобы оценивать машины новых клиентов, а клиенту нужно делать много шагов по взаимодействию с персоналом автосервиса, чтобы получить обратную связь по своему автомобилю.

Ранее этот процесс как-то улучшить не представлялось возможным, так как точная оценка повреждений требовала личного взгляда специалиста. Но сейчас, с развитием методов машинного обучения и компьютерного зрения, это стало возможным - теперь можно провести анализ изображения и параметров автомобиля с помощью искусственного интеллекта с достаточной точностью. Это позволяет определить ответы на самые частые вопросы клиентов - стоимость и длительность работ, и перенести часть работы на автоматизированный сервис.

Таким образом, целью работы является повышение эффективности процесса оценки стоимости и длительности ремонта поврежденных автомобилей.

Сейчас, при высокой конкуренции между автосервисами скорость обработки обращений является одним из важнейших факторов, влияющих на удержание и привлечение новых клиентов. Автосервис, имеющий интеллектуальную систему, которая способна практически моментально дать ответ клиенту по новому обращению, имеет преимущество перед остальными. Кли-

енту комфортнее взаимодействовать, ведь он получает обратную связь по главным интересующим вопросам почти сразу, а доверие к сервису возрастает. К тому же, на рынке отсутствуют подобного рода решения, выполняющие автоматическую оценку повреждений по фотографии автомобиля, что делает разработку инновационной и востребованной.

## 1 Обзор научно-технических источников информации

Задача поиска повреждений автомобилей совсем не нова - количество научных источников по теме автоматизации этого процесса с помощью искусственного интеллекта достаточно велико. Большинство из них описывают высокую коммерческую значимость проблемы - одни упоминают страховые компании, другие - продажи подержанных автомобилей, третьи - ремонт в автосервисах. Так, в одном из источников указано, что страховые компании США ежегодно теряют порядка 18 миллиардов долларов из-за ручной обработки обращений [1]. Эти цифры и развитие методов машинного обучения требуют внедрять решения, направленные на ускорение и упрощение оценки с помощью компьютерного зрения. В рамках рассмотренных статей исследуются методы автоматизации процесса визуальной оценки повреждений автомобилей.

В данном разделе отражены имеющиеся подходы к реализации задачи поиска и классификации повреждений по фотографии, содержащей автомобиль. Наиболее часто применяемый подход - использование разного рода архитектур свёрточных нейронных сетей (CNN).

В масштабном анализе и сводке по подходам машинного обучения к решению задачи обнаружения повреждений автомобилей, проведённом в 2025 году, подробно рассказано о применяемых методах. Их можно разделить на 2 группы - традиционные методы машинного обучения и методы, основанные на глубоком обучении [2]. Для каждой категории подходы дополнительно группируются по трем ключевым применениям: классификация, обнаружение и сегментация.

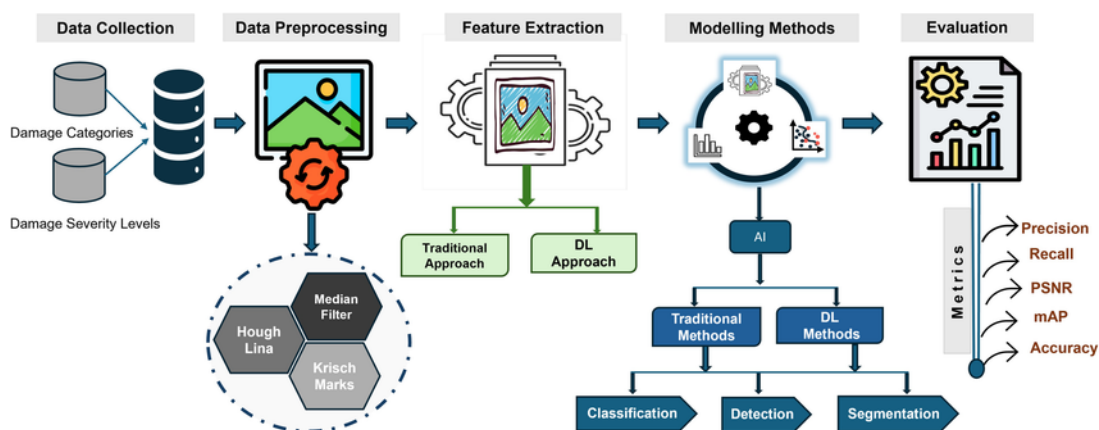


Рисунок 1 — Обзор системы обнаружения повреждений транспортных средств [2]

Классификация - для определения типа повреждения/детали, обнаружение (детекция) - для определения местоположения, а сегментация - для выделения точных областей, обеспечения точности на уровне пикселей.

Проведённый глубокий поиск статей и источников показал, что возможности традиционных методов ограничены только простыми задачами классификации, такими как различения повреждённых и неповреждённых транспортных средств или определения базовых типов повреждений. По более сложным задачам научных статей обнаружено не было. Это объясняется тем, что определение границ поврежденных областей - трудная задача для традиционных методов по причине их зависимости от низкоуровневых объектов и ограниченного пространственного контекста. [2]. В одной из статей также ссылаются на источник, в котором проводилось исследование, показавшее, что в задаче обнаружения мелких повреждений в стальных канатах свёрточные нейронные сети значительно превосходят другие методы машинного обучения, такие как kNN, SVM, Random Forest и Logistic Regression. [3]

В подавляющем большинстве рассмотренных современных работ по распознаванию и классификации повреждений транспортных средств используются CNN-архитектуры для задач классификации, детекции и сегментации. Рассматриваемые методы нацелены на быстрый и информативный анализ изображений - именно экономия времени и ресурсов при сохранении требуемого уровня точности является основной целью исследований.

## **1.1 Свёрточные нейронные сети в анализе изображений**

Глубокое обучение, в частности свёрточные нейронные сети (CNN), продемонстрировало впечатляющие результаты в широком спектре задач визуального распознавания, от обнаружения объектов до семантической сегментации [4] .

Основная идея CNN заключается в последовательном выделении признаков с изображения, начиная с простых граней, текстур на первых уровнях и заканчивая сложными формами в более глубоких слоях.

Типовая CNN состоит из последовательности сверточных слоёв, выделяющих локальные признаки изображения, и слоёв объединения (пуллинговых слоёв) для уменьшения размерности и устойчивости к смещению. После

свёрток обычно идут полносвязные слои (или сверточные «головы») для предсказания классов или координат объектов. [5]

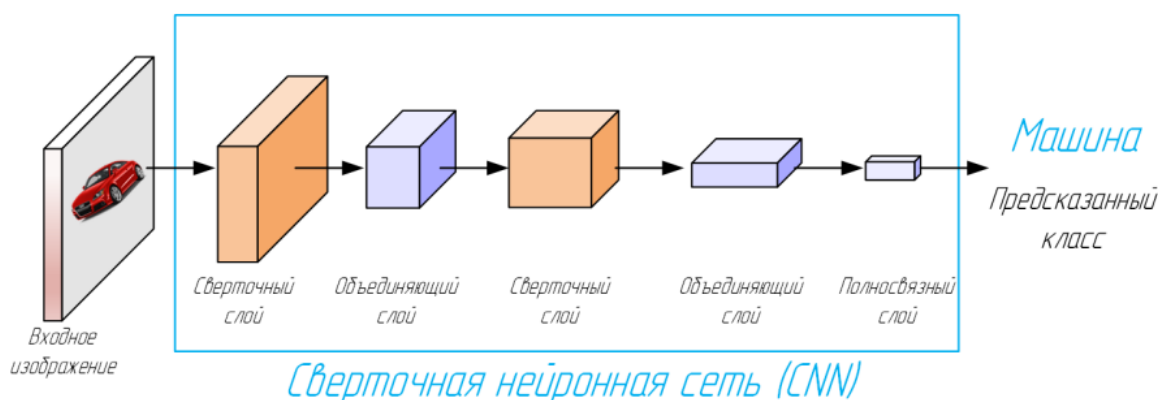


Рисунок 2 — Концепция архитектуры CNN [5]

Такая архитектура позволяет выявлять сложные визуальные закономерности на изображении. Как отмечено в одном из источников, CNN особенно хороши в распознавании краёв, форм, текстур на изображениях, что делает их отличными помощниками в классификации изображений. [1]

Современные модели с CNN архитектурой можно разбить на две большие группы - одноэтапные и двухэтапные.

Представителями двухэтапных методов является серия R-CNN. (Mask R-CNN, Faster R-CNN) Этот метод генерирует множество областей-кандидатов, которые могут содержать объекты, при первой обработке изображения. Затем для каждой области-кандидата производится повторная обработка через CNN. Это обеспечивает высокую точность предсказания, однако есть и главный недостаток: подход достаточно медленный и требует больших вычислительных ресурсов. Поэтому был разработан алгоритм YOLO (You Only Look Once), впервые предложенный в 2016 году - он обрабатывает всё изображение "за один проход что ускоряет обнаружение объектов. [6]

Серия YOLO получила широкую популярность благодаря балансу скорости и точности. Начиная с YOLOv5 и заканчивая YOLOv8, а в последнее время и YOLOv11, модели претерпели значительные архитектурные переработки, такие как улучшение базовых компонентов, подходов к агрегированию признаков и методов обучения, что в совокупности повысило эффективность обнаружения. Несмотря на улучшения и прочие достижения, в сфере обнаружения повреждений транспортных средств по-прежнему существуют проблемы. [4]



Рассмотрим более подробно существующие решения задачи оценки повреждений автомобилей.

## 1.2 Разделение задачи на подзадачи: детали, повреждения и оценка стоимости

Задача оценки повреждений достаточно масштабная. И нередко её разбивают на несколько взаимосвязанных этапов, таких как определение повреждений автомобиля и оценка стоимости. Так, в статье "Vehicle damage severity estimation for insurance operations using inthe-wild mobile images" описывается подход пайплайна подзадач: они обнаруживают автомобиль, затем проводят семантическую сегментацию всех его деталей и всех повреждённых участков, рассматривая и тот, и другой этап как задачу инстанс-сегментации с применением Mask R-CNN. После сегментации всех кузовных частей и всех повреждённых частей вычисляется относительная площадь повреждений по каждой детали, и затем передаётся в модель окончательной оценки стоимости ремонта. В работе шаг окончательной оценки стоимости и прочие шаги также выделяются как отдельные части пайплайна. [7]

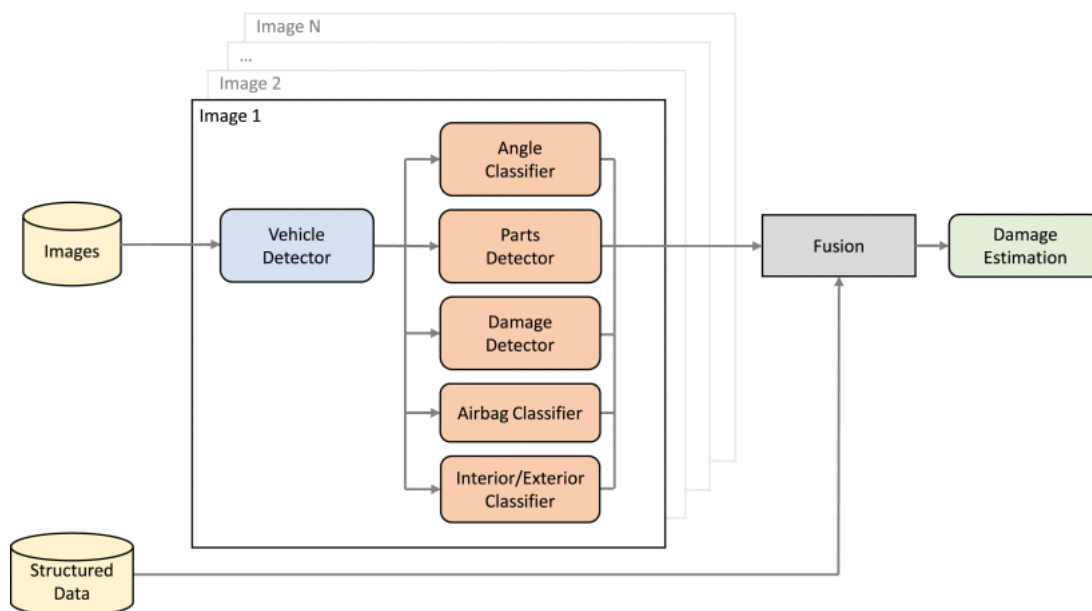


Рисунок 3 — Обзор пайплайна процесса оценки ущерба [7]

Такой подход имеет несколько преимуществ:

- Это позволяет последовательно решать проблему, поэтапно выполняя действия по получению нужного результата от анализа изображения;

- Объекты разных типов (детали и повреждения) могут требовать разных подходов для необходимой точности обнаружения, а предварительное разбиение на подзадачи сегментации позволяет снять часть фона и точнее анализировать повреждения;
- Разделение на подзадачи позволяет выполнить его с помощью нескольких моделей (например, одна модель для сегментации деталей, а вторая - для повреждений). Это позволяет сначала обучить модель находить и классифицировать детали, а затем распознавать повреждения на уже вырезанных деталях;
- Пайплайн из подзадач позволяет видеть промежуточные результаты и анализировать их. Ошибки и прочие проблемы на каждой из фаз общего процесса можно корректировать отдельно.

Продолжая тему разделения ответственности решения на несколько этапов, рассмотрим подходы по детекции и классификации как деталей, так и повреждений.

### **1.3 Подходы к распознаванию деталей автомобиля**

При анализе изображения на предмет обнаружения деталей автомобиля применяются как сегментационные, так и детекторные методы. В ряде работ реализована инстанс-сегментация деталей, позволяющая выделять деталь отдельной маской. Существует множество алгоритмов сегментации, каждый из которых имеет свои преимущества и недостатки. В статье "Evaluation of deep learning algorithms for semantic segmentation of car parts" проведено сравнение пяти алгоритмов сегментации - Mask R-CNN, Hybrid Task Cascade (HTC), Composite Backbone Network (CBNet), Path Aggregation Network (PANet) и Global Context Network (GCNet)), в котором best-performing оказался HTC с ResNet-50 по метрике mAP. Авторы работы также оценивали устойчивость модели к зашумлённым данным (в частности, при плохих погодных условиях - туман, снегопад) и GCNet показала лучший результат по метрике mPC. [8]

Параллельно с сегментацией, детали часто определяют стандартной детекцией. Так, в работе "Optimized car parts detection with advanced feature fusion and attention modules" предложена улучшенная YOLO-архитектура для детекции мелких частей автомобиля. [9]

### 1.3.1 Mask R-CNN

Алгоритм сегментации объектов Mask R-CNN был разработан на основе Faster R-CNN. Его новизна заключается в том, что Faster R-CNN мог только определять местоположение объекта на изображении и распознавать его, а Mask R-CNN также может производить и сегментацию объектов.

Mask R-CNN состоит из двух основных частей:

- Базовой модели (Backbone), которая извлекает признаки из изображения с помощью остаточной нейронной сети (ResNet), состоящей из 50–101 слоя свёрточной нейронной сети, в сочетании с сетью Feature Pyramid Network (FPN);
- "Головы" (Head), которая создаёт ограничивающую рамку (Bounding Box) вокруг интересующей области (ROI) и определяет тип объекта в этой рамке.

Ещё одна особенность, отличающая Mask R-CNN от Faster R-CNN – создание маски для каждой области интереса (ROI). [8]

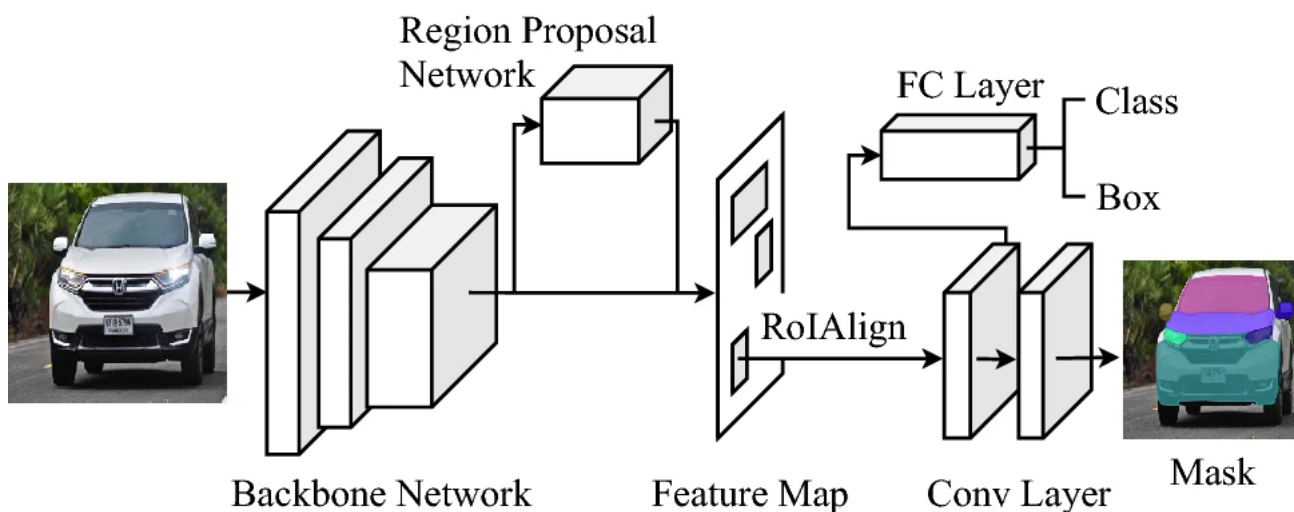


Рисунок 4 — Архитектура Mask R-CNN [8]

### 1.3.2 Global Context Network (GCNet)

Архитектура Global Context Network (GCNet) имеет структуру, схожую с Mask R-CNN, которая показана на рисунке 4. Однако особенностью GCNet является расширение Backbone блоком глобального контекста, схема которого отражена на рисунке 5.

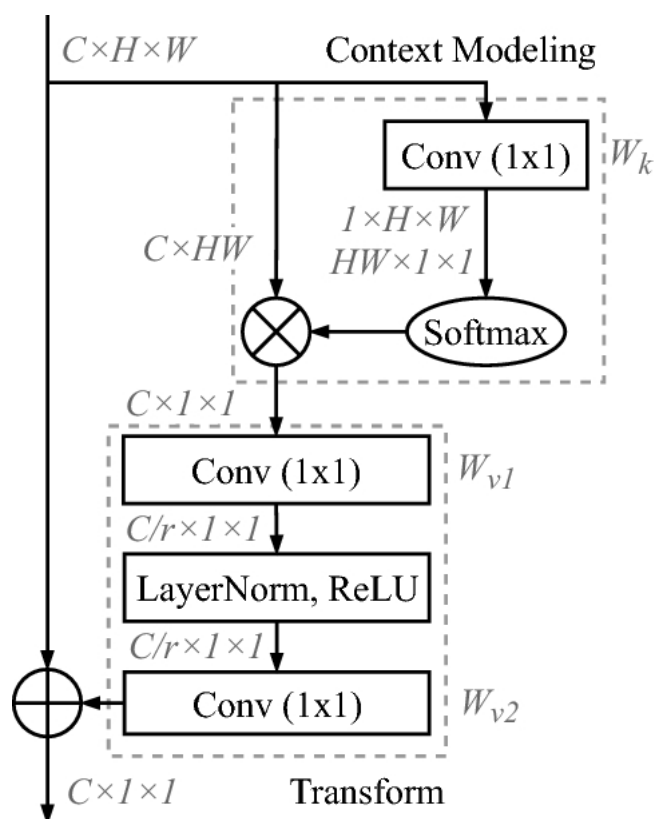


Рисунок 5 — Блок глобального контекста (Global Context). Карта признаков имеет размер  $C \times H \times W$  — количество каналов  $C$ , высота  $H$  и ширина  $W$ .  $\times$  обозначает матричное умножение, а  $+$  — поэлементное сложение с трансляцией.  $r$  — коэффициент узкого места, а  $C/r$  — размерность скрытого представления узкого места [8]

Преимущество такого блока в том, что в его состав входит Non-Local Network (NLNet), помогающая решить проблему дальнodelствующих зависимостей. [8]

### 1.3.3 Hybrid Task Cascade (HTC)

Алгоритм Hybrid Task Cascade (HTC) был создан с целью повышения эффективности инстанс-сегментации.

Особенности:

- В архитектуре блоки предсказания ограничивающих рамок (bounding box'ов), масок и извлечения ROI организованы в каскадную структуру, как показано на рисунке 6;
- Процессы предсказания ограничивающих рамок и масок выполняются параллельно и последовательно на нескольких стадиях;

- Многостадийная ветвь предсказания масок - маска, полученная на одном этапе, используется при генерации маски на следующем;
- Модель учитывает контекст при формировании каждой маски.

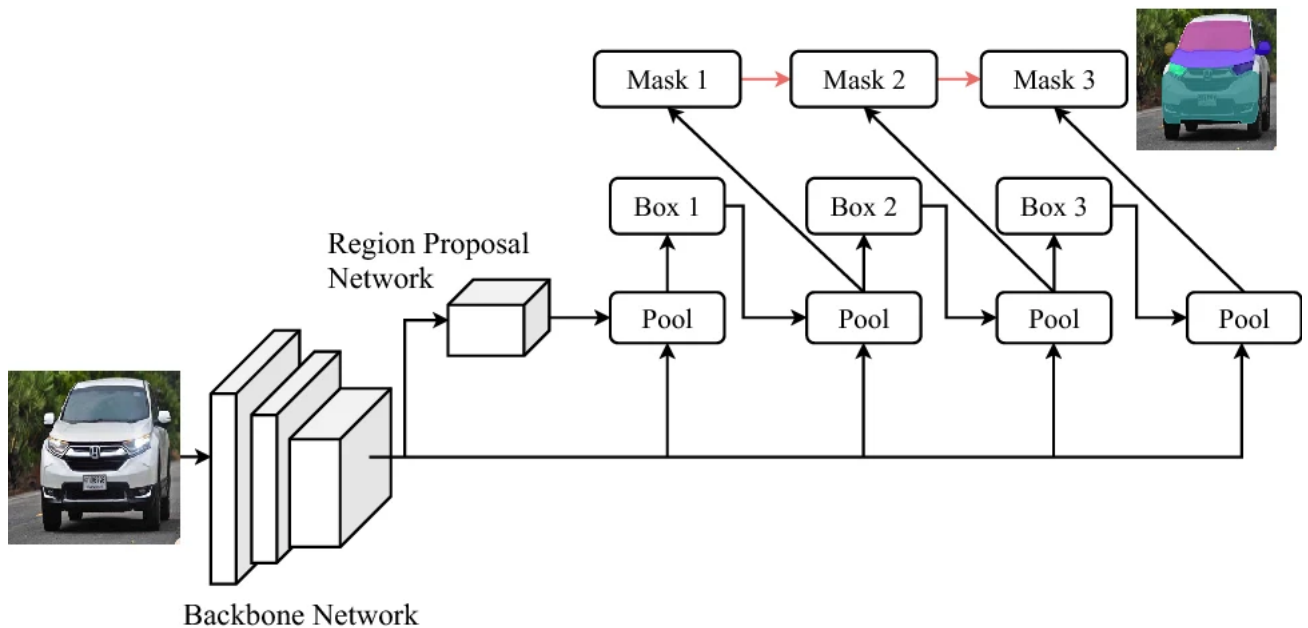


Рисунок 6 — Архитектура HTC [8]

Совокупность этих улучшений существенно улучшает передачу информации между задачами и повышает итоговую точность. [8]

#### 1.3.4 Усовершенствованная архитектура на базе YOLO

Важные особенности архитектуры:

- Внедрён C2f (Cross Stage Partial style block) блок - помогает сохранять низкоуровневые признаки;
- Внедрён модифицированный C2fCIB (C2f + Channel Interaction Block) блок - усиливает межканальные зависимости;
- PSA (Pyramid Split Attention Module) - повышает выделение мелких/схожих деталей.

Однако подход имеет и недостатки:

- Очень мелкие части (<1% изображения) всё так же остаются проблемой;
- Модель стала заметно тяжелее по параметрам (3.2М параметров у YOLOv8 против 74.6М у предложенной модели) и требует куда больше вычислений. [9]

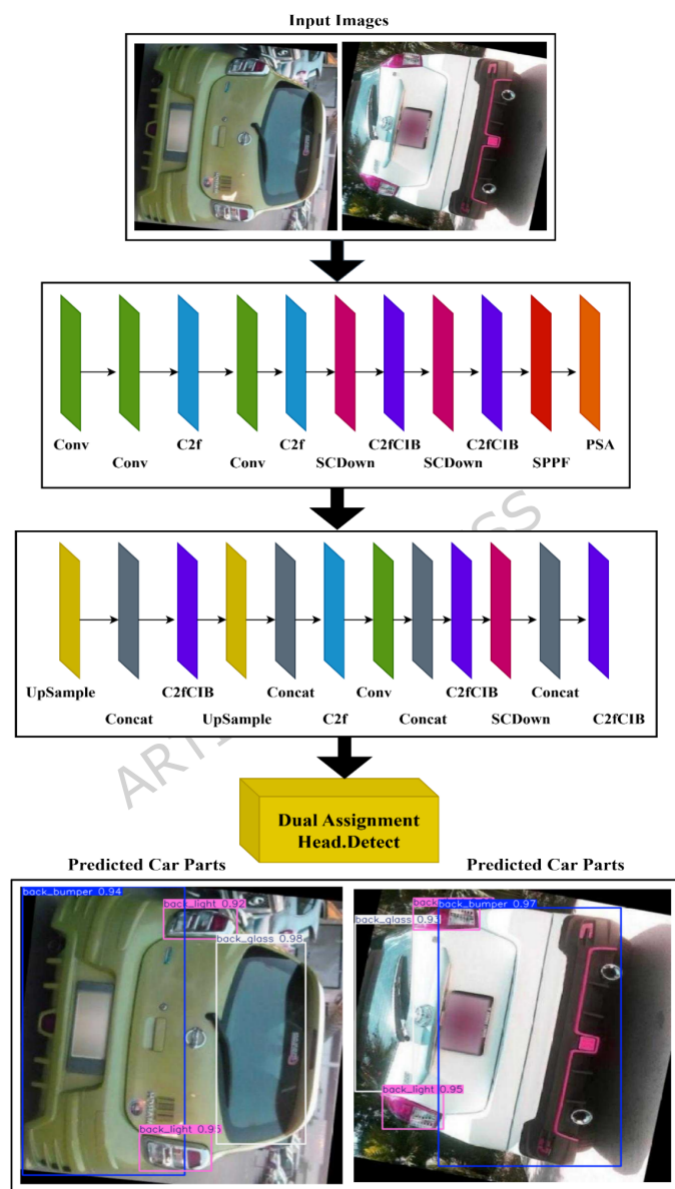


Рисунок 7 — Архитектура улучшенной YOLO [9]

Результаты сравнения с другими подходами показали, что предложенные архитектурные улучшения дают существенный рост по метрикам Recall и mAP в задаче детекции деталей автомобиля. Авторский вариант YOLO превосходит некоторые классические двухэтапные R-CNN подходы, такие как Faster R-CNN и SSD, по метрикам Precision, Recall, mAP на рассматриваемом датасете car-parts.

#### 1.4 Подходы к распознаванию повреждений автомобиля

Методы распознавания повреждений автомобиля, в отличие от задачи детекции автомобильных деталей, требуют более тщательного и точного

анализа изображений. По мнению авторов статьи "Vehicle Damage Severity Estimation for Insurance Operations Using In-The-Wild Mobile Images задача обнаружения повреждений скорее является задачей сегментации, а не задачей детекции, и многие современные методы отдают предпочтение именно сегментации. Сегментационные маски позволяют точнее описывать размытые очертания и мелкие дефекты, чем ограничивающие рамки (bounding box'ы). [7]

В статье "Automatic damaged vehicle estimator using enhanced deep learning algorithm" используют улучшенный Mask R-CNN с предварительно обученными моделями Inception-ResNetV2, VGG16, VGG19 для локализации и классификации повреждений. Комбинация Mask R-CNN с Inception-ResNetV2 показала лучшие результаты [10]

Популярны так же и решения с улучшенными YOLO-архитектурами специально для повреждений.

Один из примером улучшенного YOLO - HL-YOLO. Это модифицированный YOLOv11, использующий гетерогенные свёртки и специальные механизмы внимания.

Особенности архитектуры:

- Используются механизмы внимания для более точного выявления мелких и нерегулярных повреждений автомобиля, таких как царапины и вмятины;
- Применяется Large Separable Kernel Attention (LSKA), позволяющий учитывать пространственный контекст на больших расстояниях и повышать чувствительность к слабовыраженным, мелким дефектам;
- Дополнительным преимуществом HL-YOLO является использование модуля C3k2-HetConv, который включает гетерогенные свёртки и частичное разделение признаков. Это помогает одновременно и снизить вычислительные затраты, и повысить чувствительность к мелким повреждениям. [4]

Также в источнике "Small Dents, Big Impact: A Dataset and Deep Learning Approach for Vehicle Dent Detection" был проведён следующий эксперимент: собран датасет мелких вмятин и обучены YOLOv8-модели. Модель YOLOv8m-t42 достигла точности 0.86 и полноты 0.84 при обнаружении даже микроскопических вмятин. Полученные результаты указывают на то, что од-

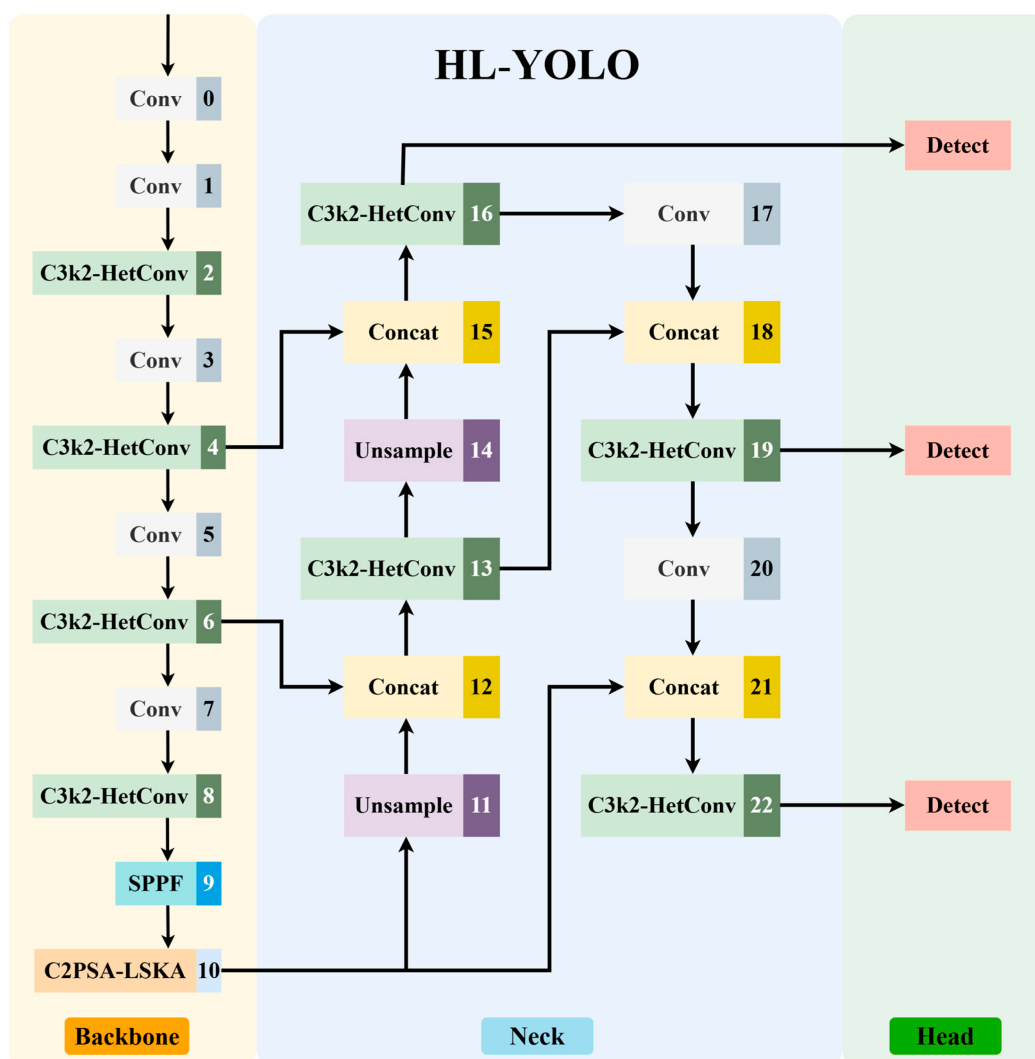


Рисунок 8 — Архитектура HL-YOLO [4]

ностадийные детекторы тоже могут быть эффективны для этой узкой задачи нахождения повреждений. [11]

Проведённые сравнения моделей говорят о том, что выбор зависит от требований к точности и скорости. В одном из сравнений YOLO и Faster R-CNN показано, что YOLO показывают результат по метрикам Recall и mAP немного хуже, чем Faster R-CNN, однако требуют меньше ресурсов (GFLOPs) и намного быстрее (FPS). [4]

| Algorithm    | Precision/% | Recall/% | mAP50/% | mAP50-95/% | GFLOPs | Model Size/MB | FPS   |
|--------------|-------------|----------|---------|------------|--------|---------------|-------|
| DETR         | 63.0        | 60.7     | 63.8    | 33.0       | 36.8   | 36.7          | 75.8  |
| Faster R-CNN | 68.2        | 60.0     | 65.0    | 34.1       | 37.5   | 28.2          | 84.7  |
| YOLOv5n      | 65.0        | 53.7     | 60.7    | 31.1       | 7.1    | 2.5           | 294.1 |
| YOLOv8n      | 65.2        | 54.5     | 61.9    | 31.1       | 8.1    | 3.0           | 277.8 |
| YOLO11n      | 70.8        | 53.6     | 61.9    | 32.1       | 6.3    | 2.6           | 227.3 |
| HL-YOLO      | 72.6        | 56.7     | 64.3    | 33.1       | 7.4    | 2.7           | 188.7 |

Рисунок 9 — Точность и эффективность различных моделей обнаружения [4]



Исследование показывает, что детекторы YOLO дают баланс скорости и точности, в то время как инстанс-сегментаторы типа Mask R-CNN могут обеспечивать более высокую точность (маски повреждений), но с использованием больших ресурсов.

Таблица 1 — Архитектурные подходы к решению задачи оценки повреждений

| Описание архитектурного подхода                             | Минусы                                        | Плюсы                                                                     |
|-------------------------------------------------------------|-----------------------------------------------|---------------------------------------------------------------------------|
| Общая модель и для деталей, и для повреждений               | Сложнее обучать и интерпретировать результаты | Минимальное кол-во этапов обработки                                       |
| Пайплайн подзадач (детали и повреждения как отдельные шаги) | Увеличение числа этапов                       | Улучшенная управляемость, точность, возможность оптимизации каждого этапа |

Таблица 2 — Подходы(модели) для сегментации деталей автомобиля

| Описание подхода / модели      | Минусы                                           | Плюсы                                                            |
|--------------------------------|--------------------------------------------------|------------------------------------------------------------------|
| Mask R-CNN                     | Высокая сложность, скорость инференса ниже       | Очень высокая точность масок, чёткое разделение деталей          |
| Hybrid Task Cascade (HTC)      | Более сложная архитектура и обучение             | Достаточно высокая точность - за счёт каскадного уточнения масок |
| Global Context Network (GCNet) | Увеличение сложности модели                      | Устойчивость к нестандартным условиям съёмки, шумам              |
| YOLO-архитектуры               | Могут выдавать недостаточно точные маски деталей | Высокая скорость обучения и инференса                            |

Таблица 3 — Подходы(модели) для сегментации повреждений автомобиля

| Описание модели             | Минусы                                                   | Плюсы                                         |
|-----------------------------|----------------------------------------------------------|-----------------------------------------------|
| Mask R-CNN                  | Требует точной разметки, много ресурсов, дольше инференс | Точно выделяет повреждения, лучше локализация |
| YOLOv8                      | Может выдавать плохие результаты на сложных повреждениях | Баланс скорости и точности результатов        |
| Улучшенные YOLO-архитектуры | Недостаточно точные границы повреждений                  | Повышенная чувствительность к мелким дефектам |

Таблица 4 — Сравнение двухэтапных и одноэтапных подходов

| Подход                              | Минусы                                                | Плюсы                                     |
|-------------------------------------|-------------------------------------------------------|-------------------------------------------|
| Двухэтапные (Mask R-CNN, HTC и др.) | Медленные, требовательны к ресурсам и точной разметке | Максимальная точность сегментации         |
| Одноэтапные (YOLO и модификации)    | Менее точные маски                                    | Высокая скорость и более простое обучение |

## 2 Структурный системный анализ исследуемого объекта

Объектом исследования в данной работе является процесс оценки стоимости ремонта автомобилей в автосервисе. Процесс достаточно обширен и включает в себя множество действий, начиная от обращения клиента и заканчивая принятием решения о начале ремонтных работ (либо отказом). В него вовлечены три основные роли:

- Клиент;
- Сотрудник автосервиса (мастер);
- Начальник автосервиса.

Для погружения в контекст процесса и понимания его деталей, была составлена BPMN-диаграмма, подробно отражающая состояние "as is" в автосервисе. (Рисунок 10)

Проанализировав диаграмму, можно выявить основные проблемные места процесса:

- Не все клиенты обращаются в автосервис лично. Многие делают это в режиме онлайн, и этап сбора информации сопряжён с дополнительными трудностями. Иногда может быть недоступен сотрудник автосервиса, к которому обращаются, и клиенту приходится ожидать, пока мастер дойдёт до устройства, обработает обращение и даст обратную связь;
- Несогласованный формат отправки изображений мастеру приводит к тому, что иногда информации с изображений недостаточно (например, не понятно, что за деталь), и приходится запрашивать уточнение данных или новое фото. Это приводит к затратам времени на переписку (дополнительные циклы пересъёмки/разъяснений, ожиданий ответа с обеих сторон);
- Иногда неопытность сотрудника и отсутствие чёткого понимания расценки работ приводит либо к необходимости согласования с начальником автосервиса (дополнительное время на согласование), либо к тому, что одна и та же поломка может оцениваться по-разному разными мастерами.

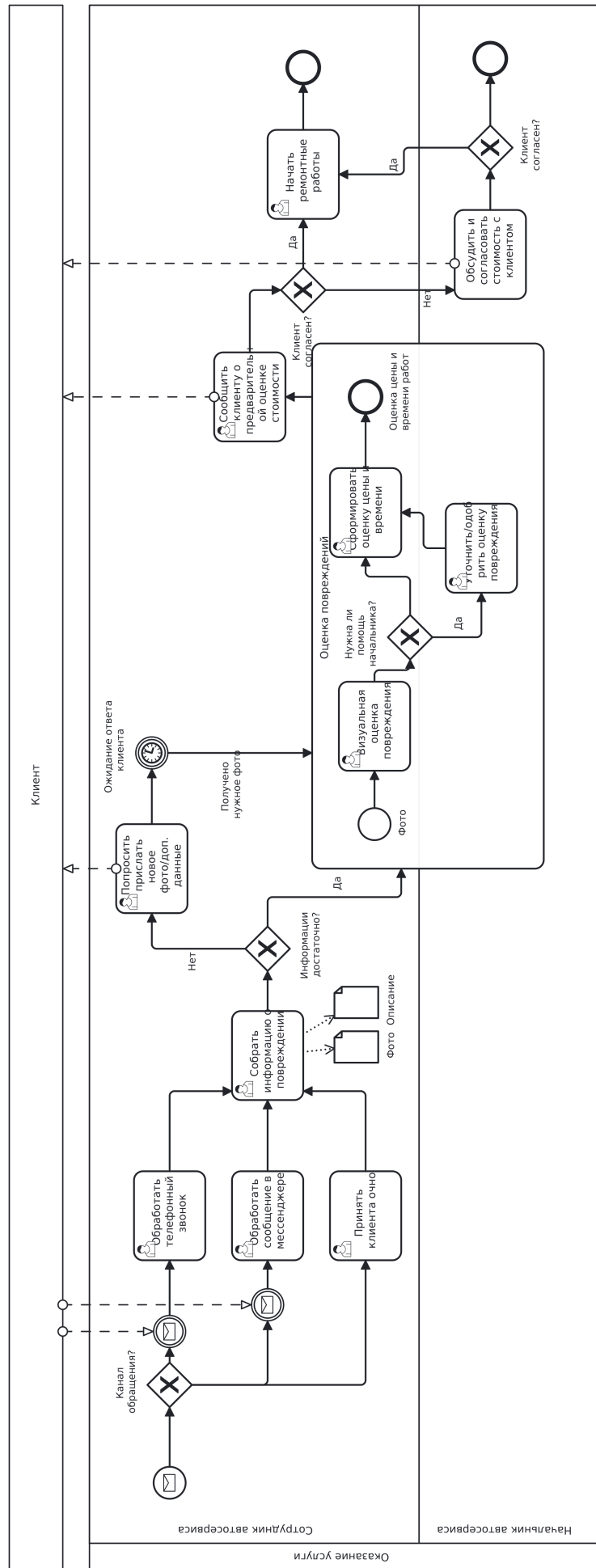


Рисунок 10 — BPMN-диаграмма процесса

По экспертной информации, предоставленной начальником автосервиса, более половины случаев обрабатываются в режиме онлайн, а время обработки одного онлайн-обращения занимает в среднем 30 минут. Это указывает на высокую загруженность мастеров в дни обработки обращений сразу от множества клиентов. Около 20% обращений обсуждаются с начальником, а с клиентом контакты осуществляются в среднем 2-3 раза, что также говорит о весомых тратах времени на согласование.

Таким образом, в текущем бизнес-процессе имеется несколько существенных недостатков:

- Длительные согласования и ожидания ответа;
- Отвлечение мастеров от работы на приём обращений;
- Ошибки при оценке стоимости у новых сотрудников.

По имеющейся статистике, вероятность существенного изменения (более, чем на 5-10%) предварительной оценки стоимости ремонта - всего около 5%. Это указывает на устойчивость и воспроизводимость результатов предварительной оценки.

### 3 Постановка задачи

Структурный системный анализ объекта исследования показал, что значительная доля заявок клиентов поступает в виде фотографий, а процесс обработки обращений имеет множество недостатков, таких как длительные ожидания и согласования, просчёты новых сотрудников. Устойчивость и воспроизводимость результатов предварительной оценки подтверждает возможность и необходимость внедрения автоматизированного сервиса оценки повреждений автомобилей по фотографии.

Цель работы - повышение эффективности процесса оценки стоимости ремонта поврежденных автомобилей. Был сформулирован список задач, которые необходимо выполнить для достижения цели:

1. Сформировать датасет для обучения модели путём сбора и фильтрации изображений повреждённых автомобилей и их дальнейшей разметки.
2. Разработать модули для обработки и аугментации изображений, устранения недостатков съёмки
3. Обучить ML-модель обнаружения и классификации деталей и повреждений на основе размеченных данных
4. Реализовать бизнес-логику оценки стоимости и длительности ремонта
5. Спроектировать и реализовать микросервисную архитектуру приложения, описать и произвести процесс интеграции с ботом в мессенджере
6. Описать и разработать архитектуру базы данных
7. Интеграция CI/CD и процесса поставки программного обеспечения
8. Провести тестирование работоспособности системы

Предполагаемым результатом работы является реализованный сервис с внедрённой в него ML-моделью распознавания и классификации повреждений автомобиля.

Эффективность решения можно будет оценить двум основным факторам:

1. Производительность системы
  - Время обработки одного изображения;
  - Пропускная способность сервисов (RPS).

2. Минимизация человеческого участия - на сколько уменьшится время обработки обращений клиентов

## 4 Архитектурно-техническое решение задачи

При проведении структурного системного анализа была создана модель “as is“, которая показала, что значительная часть работы выполняется вручную. Мастера автосервиса тратят большое количество времени на общение с клиентом в мессенджере, анализ фотографий и получение информации, формирование предварительной оценки. Такой подход требует лишних временных затрат и увеличивает нагрузку на сотрудников.

Чтобы решить описанную выше проблему, рассмотрим внедрение интеллектуальной системы в текущий процесс на BPMN-диаграмме "to be". (Рисунок 11). В диаграмме значительная часть операций текущего процесса передаётся новой автоматизированной системе. Функционал, предоставляемый системой для пользователя, должен включать:

1. Начало взаимодействия с системой через мессенджер. Пользователь имеет возможность написать боту и выбрать режим работы системы
2. Выбор способа указания повреждений автомобиля. Система должна обеспечивать возможность выбрать один из двух способов указания повреждений:
  - Указать повреждения и их типы через интерфейс бота;
  - Отправить фотографию повреждённого автомобиля, которую автоматически проанализирует система.
3. Если пользователь выбрал автоматический режим:
  - Система должна автоматически выполнить обработку и анализ фотографии с использованием методов машинного обучения: определить видимые детали автомобиля, имеющиеся на них повреждения и сформировать аннотированное изображение с выделенными масками деталей и повреждений. Должны обнаруживаться следующие детали:
    - Дверь;
    - Переднее крыло;
    - Заднее крыло;
    - Бампер;
    - Багажник;
    - Капот;
    - Фара;



- Переднее стекло;
- Заднее стекло;
- Боковое стекло;
- Шины и в целом колёса.

Должны обнаруживаться следующие повреждения:

- Царапина;
  - Вмятина;
  - Отвалившийся кусок краски;
  - Ржавчина;
  - Трещина;
  - Разрывы/разрезы/дырки;
  - Разбитые стекло или фары;
  - Спущенные шины и любые проблемы с колёсами.
- Пользователь должен иметь возможность посмотреть результат автоматического анализа перед переходом к этапу расчёта стоимости ремонта;
  - Пользователь должен иметь возможность отредактировать результат анализа через интерфейс бота, если он не согласен с результатом, выданным моделью:
    - Изменить тип повреждения;
    - Изменить принадлежность повреждения к детали автомобиля;
    - Удалить ошибочно найденное повреждение;
    - Добавить новое необнаруженное повреждение.
4. После проверки пользователем всех параметров повреждений система должна предоставить ему возможность подтвердить итоговый список повреждений автомобиля, чтобы перейти к расчёту.
  5. После подтверждения от пользователя система должна приступить к автоматическому расчёту ориентировочных стоимости и длительности работ в автосервисе согласно заданным параметрам, сопоставив их с правилами расчёта автосервиса (Таблицы 5 и 6)
  6. Пользователь должен получить сообщение с результатами расчёта, включающими стоимость и длительность ремонтных работ.

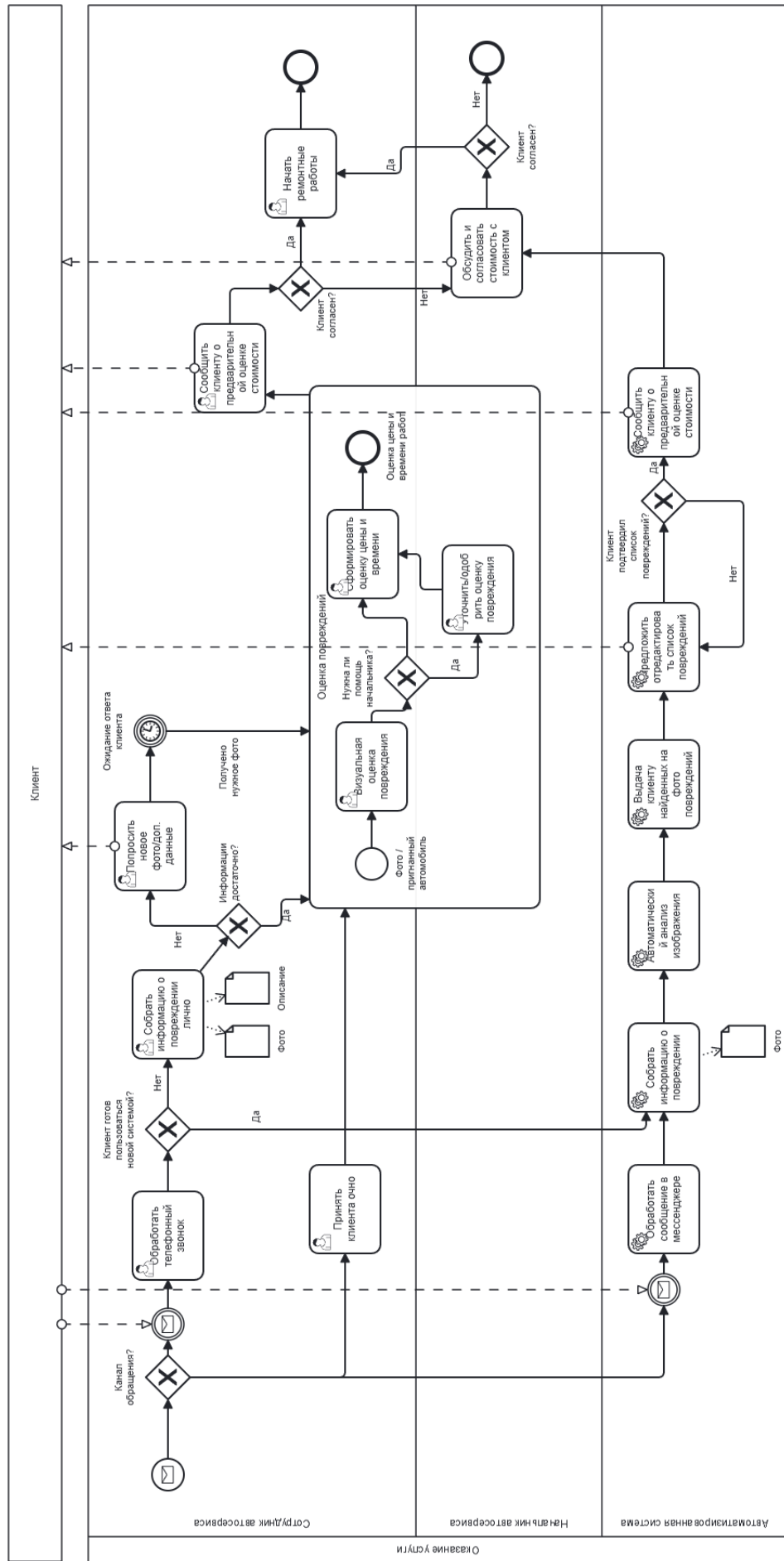


Рисунок 11 — BPMN-диаграмма ТО ВЕ

Таблица 5 — Стоимость ремонта, тыс. руб.

| Деталь           | Царапина<br>(полировка) | Царапина<br>(покраска) | Вмятина<br>(рихтовка + покраска) | Замена детали |
|------------------|-------------------------|------------------------|----------------------------------|---------------|
| Переднее крыло   | 1                       | 10–18                  | 23–30                            | 20            |
| Заднее крыло     | 1                       | 10–18                  | 23–30                            | 75–100        |
| Дверь            | 1                       | 10–18                  | 23–30                            | 20            |
| Крышка багажника | 1                       | 10–18                  | 23–30                            | 20            |
| Крыша            | 1                       | 10–18                  | 23–30                            | 75–100        |
| Капот            | 1                       | 10–18                  | 30–35                            | 28–30         |
| Бампер           | 1                       | 10–18                  | 3–5                              | 18            |
| Лобовое стекло   | —                       | —                      | —                                | 5–10          |
| Боковое стекло   | —                       | —                      | —                                | 3             |
| Фара             | —                       | —                      | —                                | 3             |

Таблица 6 — Длительность ремонта

| Деталь           | Царапина<br>(полировка) | Царапина<br>(покраска) | Вмятина<br>(рихтовка + покраска) | Замена детали |
|------------------|-------------------------|------------------------|----------------------------------|---------------|
| Переднее крыло   | 1 час                   | 1 день                 | 2–3 дня                          | 1 день        |
| Заднее крыло     | 1 час                   | 1 день                 | 2–3 дня                          | 5 дней        |
| Дверь            | 1 час                   | 1 день                 | 2–3 дня                          | 1.5–2 дня     |
| Крышка багажника | 1 час                   | 1 день                 | 2–3 дня                          | 1.5–2 дня     |
| Крыша            | 1 час                   | 1 день                 | 2–3 дня                          | 5 дней        |
| Капот            | 1 час                   | 1 день                 | 2 дня                            | 2 дня         |
| Бампер           | 1 час                   | 1 день                 | 1–2 дня                          | 1.5 дня       |
| Лобовое стекло   | —                       | —                      | —                                | 1 день        |
| Боковое стекло   | —                       | —                      | —                                | 1 день        |
| Фара             | —                       | —                      | —                                | 0.5 дня       |

После выполнения всех шагов выше клиент сможет проще принять решение о пользовании услугами автосервиса, и сделать лишь финальный звонок сотруднику автосервиса (о времени визита / согласовании цены).

Предлагаемое решение, встроенное в текущий процесс, имеет несколько преимуществ:

1. При поступлении обращения через канал мессенджера можно полностью передать управление системе - клиенту достаточно будет отправить фотографию или самостоятельно указать все имеющиеся повреждения.

2. Сотруднику автосервиса не требуется проводить долгое общение в мессенджере для формирования ответа по стоимости - в системе клиент это может сделать самостоятельно, и, благодаря удобному интерфейсу, быстро задать параметры и получить ответ.
3. Помимо полной автоматизации процесса при поступлении обращений через мессенджеры, можно предлагать клиентам пользоваться системой и при поступлении телефонных звонков - это также будет кратно экономить время.

Для реализации функционала необходимо разработать приложение, которое будет обеспечивать описанные сценарии. Архитектура и математическая модель системы будут рассмотрены в следующем разделе.

## 5 Математическая модель решения задачи

### 5.1 Архитектура решения в нотации С4

Перед началом разработки приложения рассмотрим архитектуру будущего решения в виде нотации С4 - она представлена на рисунках 12-15.

На рисунке 12 изображен уровень системного контекста - здесь отражено взаимодействие клиента с системой. Посредником в этом процессе является мессенджер, через интерфейс которого пользователю будет удобно взаимодействовать с системой.

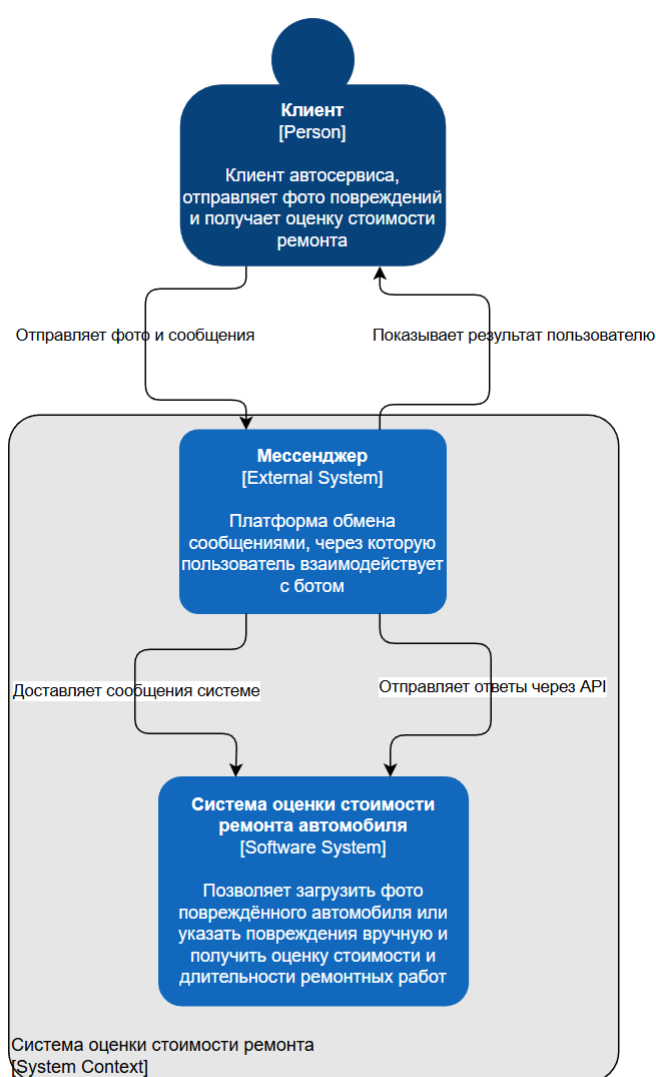


Рисунок 12 — Уровень системного контекста в нотации С4

На рисунке 13 представлен контейнерный уровень. Здесь раскрываются подробности внутренней архитектуры системы оценки стоимости ремонтных

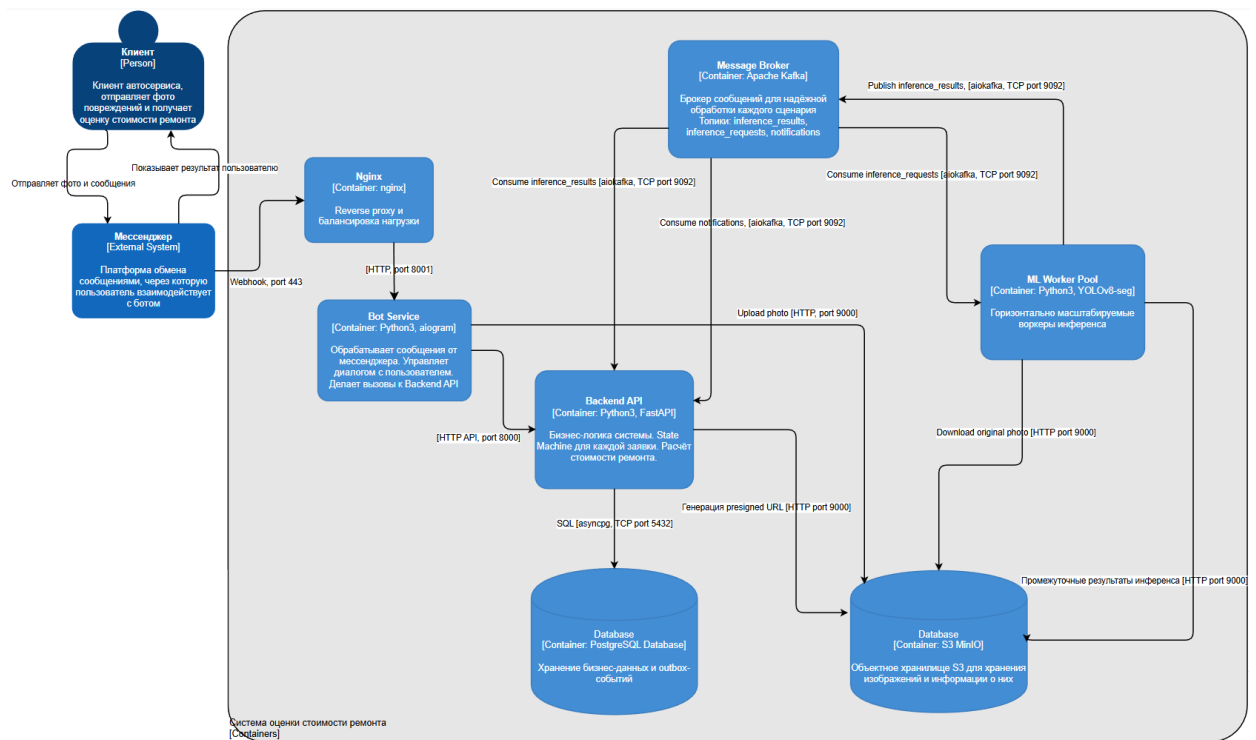


Рисунок 13 — Уровень контейнеров в нотации C4

работ. На схеме показана информация о следующих элементах системы и их взаимодействии:

- Bot Service - принимает сообщения от пользователя мессенджера, преобразует их, отправляет системе и возвращает ответы пользователю;
- Backend API - главный серверный компонент, в котором сосредоточена основная бизнес-логика:
  - Управление сценарием обработки обращений и расчёт стоимости работ;
  - Взаимодействие с базами данных;
  - Взаимодействие с брокером сообщений.
- Nginx - обратный прокси-сервер, принимающий входящие запросы. Балансировщик нагрузки;
- Message Broker (Kafka) - брокер сообщений, обеспечивающий отказоустойчивое взаимодействие между бэкендом и ML;
- ML Worker Pool - пул обработчиков для анализа изображений;
- PostgreSQL - реляционная база данных для хранения служебных данных - информации о сценариях, статусах, результатах и др.;
- MinIO S3 - объектное хранилище для хранения исходных и аннотированных изображений.



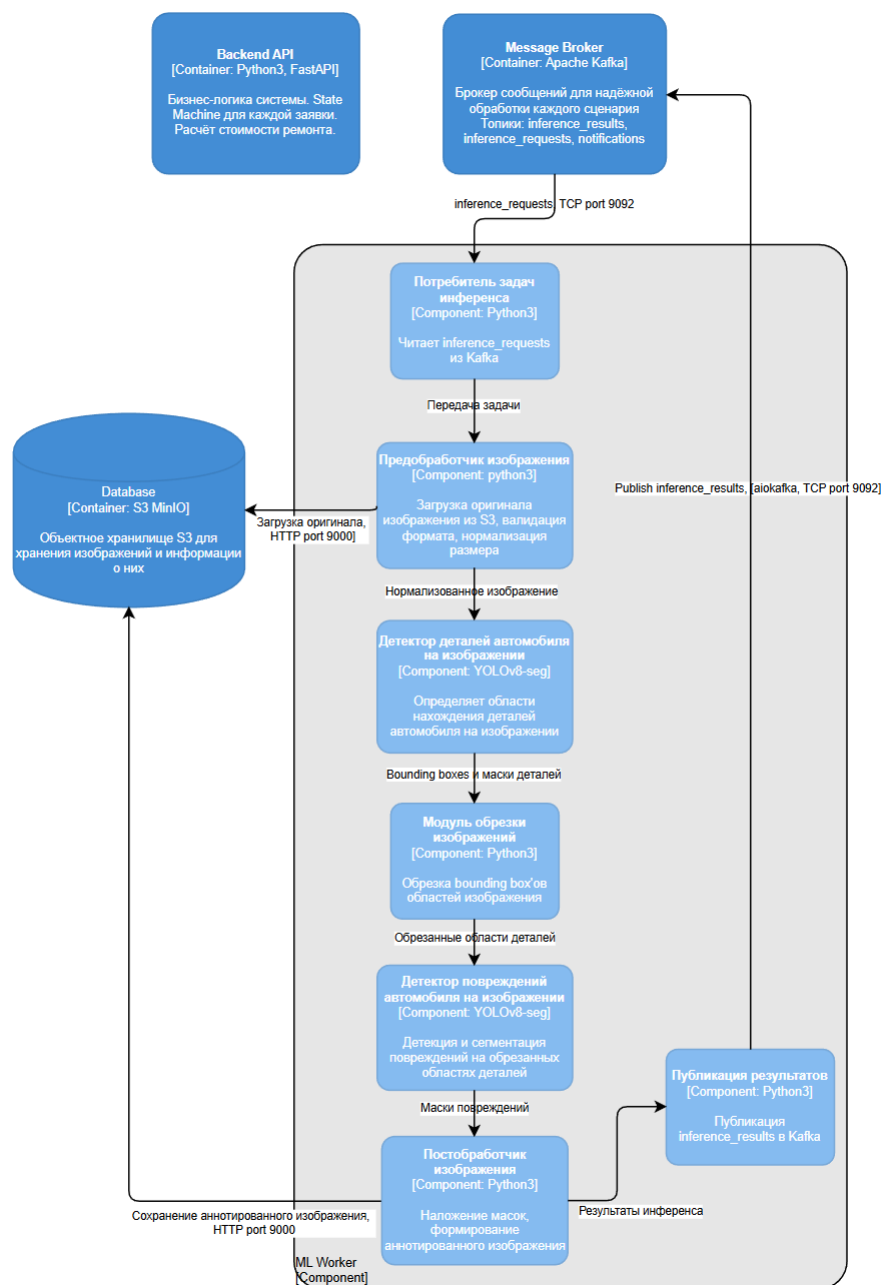


Рисунок 15 — Компонентный уровень C4 - контейнер ML Worker

Детальный обзор ML контейнера представлен на рисунке 15. Он получает сообщение от брокера сообщений о появившихся сценариях и производит цикл действий по обработке фотографии вплоть до формирования финального аннотированного изображения, которое сохраняет в S3. ML Worker состоит из следующих компонентов:

- Предобработчик изображений - отвечает за подготовку изображения к анализу;
- Детектор деталей автомобиля на изображении - определяет области расположения деталей;



- Модуль обрезки изображений - отвечает за обрезку отдельных фрагментов изображения (деталей) по найденным областям;
- Детектор повреждений автомобиля на изображении - определяет области расположения повреждений на обрезанных областях деталей;
- Постобработчик изображения - формирует итоговое размеченное изображение.

## 5.2 Схема базы данных

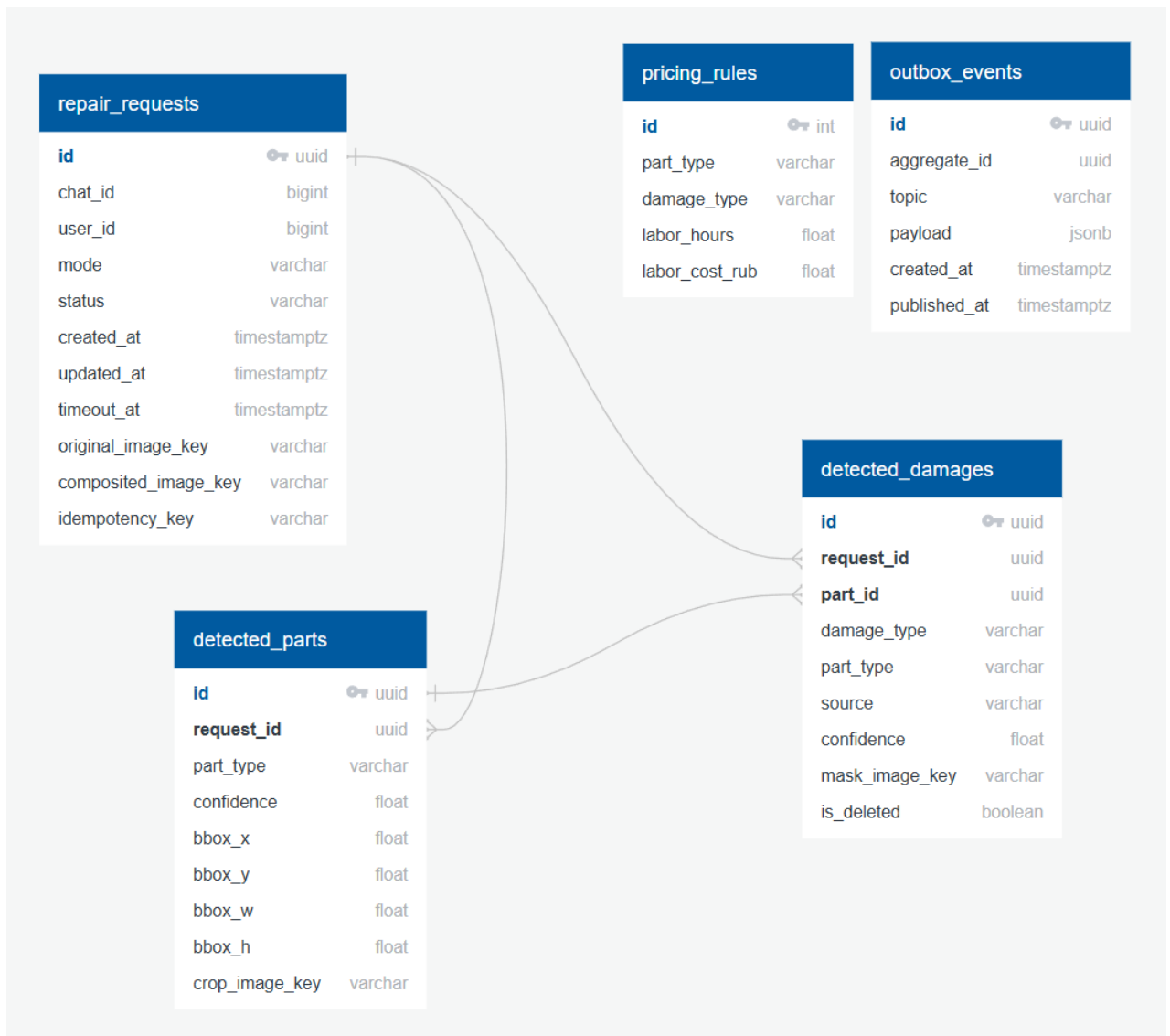


Рисунок 16 — Схема базы данных PostgreSQL

В системе имеется два хранилища данных - S3 MinIO для изображений и PostgreSQL для хранения данных о каждом сценарии и изображении. Рассмотрим схему базы данных PostgreSQL подробнее. Она создаётся скриптом инициализации при первом запуске контейнера и содержит несколько таблиц:

- `repair_requests` - хранение информации о заявке на оценку стоимости: информация о чате с пользователем, статусе, режиме обработки, изображении;
- `detected_parts` - информация о результате сегментации деталей - тип детали, уверенность, ограничивающий прямоугольник;
- `detected_damages` - информация о найденных повреждениях - тип повреждения, уверенность, маска на изображении;
- `pricing_rules` - таблица для этапа расчёта стоимости и длительности работ на основе параметров <деталь - повреждение>;
- `outbox_events` - доменные события для Kafka.

Визуализированная схема базы данных PostgreSQL с показанными связями и типами данных представлена на рисунке 16.

### 5.3 Архитектура и математическая модель ML-составляющей

Рассмотрим более подробно архитектуру контейнера ML Worker. Процесс внутри представлен в виде пайплайна подзадач - общий процесс детекции и сегментации повреждений разделён на несколько этапов, главными из которых являются компоненты, требующие обучения моделей детекции и сегментации деталей и повреждений (Причины выбора такого подхода описаны в главе 1).

Для решения задачи анализа изображения целесообразно выбрать модель семейства YOLO. Полноценное сравнение и анализ архитектур моделей производились в главе 1, и текущий выбор обусловлен тем, что YOLOv8 представляет собой компромисс между скоростью обучения и внедрения, скоростью инференса, вычислительной эффективностью и качеством предсказания.

В обобщённом виде архитектуру YOLOv8-seg можно представить как совокупность трёх частей:

- `backbone` - извлечение признаков из входного изображения;
- `neck` - объединение признаков разных масштабов;
- `segment head` - предсказание классов, координат объектов и коэффициентов масок.

Для обучения модели используются изображения повреждённых автомобилей. Подготовка данных и обучение поддерживают механизмы аугмента-

ции, включая Mosaic - преобразование, при котором несколько изображений объединяются в одно для улучшения обобщающей способности модели.

### 5.3.1 Модель детекции и сегментации деталей автомобиля

На вход системы поступает изображение автомобиля:

$$X \in \mathbb{R}^{H \times W \times 3},$$

где  $H$  — высота изображения,  $W$  — ширина изображения, а третье измерение соответствует цветовым каналам.

Перед передачей изображения в модель выполняется предобработка: изображение приводится к размеру входа модели 640 x 640 пикселей, а значения пикселей масштабируются к диапазону  $[0,1]$  путём деления на 255. Согласно документации Ultralytics [12], такая схема соответствует стандартному способу подготовки данных для YOLO-моделей.

Первый этап пайплайна - изображение автомобиля обрабатывается моделью детекции и сегментации деталей. Обученная модель сегментации отображает входное изображение в множество предсказаний, каждое из которых содержит bounding box, маску, класс и confidence.

Результатом работы модели деталей является множество:

$$S_{\text{parts}} = \{(b_i, m_i, p_i, s_i)\}_{i=1}^N,$$

где:

- $b_i$  - bounding box (ограничивающий прямоугольник) детали в координатах исходного изображения;
- $m_i$  - маска детали;
- $p_i$  - класс детали автомобиля;
- $s_i \in [0,1]$  - значение confidence (уверенности модели в предсказании);
- $N$  - количество найденных деталей.

Для каждой обнаруженной на фотографии детали модель оценивает и выдаёт вероятность принадлежности к одному из классов: дверь, переднее или заднее крыло и другим - полный список деталей был описан в главе 4.

После получения предсказаний выполняется фильтрация по порогу уверенности:

$$S'_{\text{parts}} = \{(b_i, m_i, p_i, s_i) \in S_{\text{parts}} \mid s_i \geq \tau_{\text{parts}}\},$$

где  $\tau_{\text{parts}}$  - порог уверенности модели в предсказании.

Для устранения дублирующихся и пересекающихся предсказаний применяется non-maximum suppression (NMS). Для двух bounding box-ов  $b_i$  и  $b_j$  коэффициент пересечения определяется как Intersection over Union (IoU) - мера перекрытия двух областей, которая определяется отношением площади их пересечения к площади объединения:

$$IoU(b_i, b_j) = \frac{|b_i \cap b_j|}{|b_i \cup b_j|}.$$

Если

$$IoU(b_i, b_j) > \lambda,$$

то предсказание с меньшим значением confidence отклоняется. NMS применяется отдельно для каждого класса объектов.

Обучение модели деталей выполняется с использованием комбинированной функции потерь, подробнее описанной в разделе 6.2.3.

### 5.3.2 Формирование обрезанных областей деталей автомобиля

После детекции, сегментации и фильтрации деталей выполняется построение кропов (обрезанных областей изображения). Для каждой детали из полученного списка выбирается ограничивающий прямоугольник  $b_i$ , по которому из исходного изображения вырезается соответствующая область. На этом этапе формируется локальная область изображения в виде прямоугольника, содержащая только одну деталь автомобиля. Такой подход снижает влияние фона и повышает точность поиска повреждений, а также позволяет рассматривать каждую деталь как отдельный объект анализа.

### 5.3.3 Модель детекции и сегментации повреждений

Для каждой детали автомобиля выполняется отдельный этап поиска повреждений. На вход второй модели поступает не полное изображение автомобиля, а кроп детали. Индекс  $i$  обозначает номер детали, для которой выполняется анализ: модель применяется независимо к каждому кропу.

Результат работы модели для  $i$ -й детали можно записать как:

$$S_{\text{damages},i} = \{(b_{ij}, m_{ij}, d_{ij}, s_{ij})\}_{j=1}^{M_i},$$

где:

- $b_{ij}$  - bounding box (ограничивающий прямоугольник) повреждения в координатах кропа детали;
- $m_{ij}$  - маска повреждения;
- $d_{ij}$  - класс повреждения;
- $s_{ij}$  - значение confidence (уверенности модели в предсказании);
- $M_i$  - количество найденных повреждений на  $i$ -й детали.

Общее множество повреждений по поданному изображению формируется как объединение результатов по всем кропам:

$$S_{\text{damages}} = \bigcup_{i=1}^{N'} S_{\text{damages},i},$$

где  $N'$  — количество деталей, прошедших фильтрацию на первом этапе.

Для обучения модели повреждений используется тот же класс архитектуры YOLOv8-seg, однако датасет строится уже на изображениях кропов деталей, преимущества такого подхода были описаны выше.

Для повреждений также используется комбинированная функция потерь, включающая локализацию, классификацию, распределительную регрессию и сегментацию:

$$\mathcal{L} = \lambda_{\text{box}} \mathcal{L}_{\text{box}} + \lambda_{\text{cls}} \mathcal{L}_{\text{cls}} + \lambda_{\text{dfl}} \mathcal{L}_{\text{dfl}} + \lambda_{\text{seg}} \mathcal{L}_{\text{seg}},$$

где:

- $\mathcal{L}_{\text{box}}$  - ошибка локализации bounding box;
- $\mathcal{L}_{\text{cls}}$  - ошибка классификации;
- $\mathcal{L}_{\text{dfl}}$  - Distribution Focal Loss;
- $\mathcal{L}_{\text{seg}}$  - ошибка сегментации масок;
- $\lambda_{\text{box}}, \lambda_{\text{cls}}, \lambda_{\text{dfl}}, \lambda_{\text{seg}}$  - веса соответствующих компонент.

Complete IoU (CIoU) расширяет IoU тремя дополнительными штрафными членами:

$$CIoU(b_{\text{pred}}, b_{\text{gt}}) = IoU(b_{\text{pred}}, b_{\text{gt}}) - \frac{\rho^2(c_{\text{pred}}, c_{\text{gt}})}{d^2} - \alpha v,$$

где  $\rho(c_{\text{pred}}, c_{\text{gt}})$  - евклидово расстояние между центрами предсказанного и эталонного прямоугольников,  $d$  - диагональ минимального охватывающего прямоугольника, а  $v$  - штраф за несоответствие соотношений сторон, взвешенный коэффициентом  $\alpha$ .

Компонент локализации можно записать как:

$$\mathcal{L}_{\text{box}} = 1 - CIoU(b_{\text{pred}}, b_{\text{gt}}),$$

где  $b_{\text{pred}}$  и  $b_{\text{gt}}$  - предсказанный и эталонный bounding box соответственно.

### 5.3.4 Формирование финального аннотированного изображения

После получения результатов на всех кропах необходимо выполнить перенос найденных масок обратно на исходное изображение.  $m_1, m_2, \dots, m_k$  - маски всех обнаруженных повреждений, приведённые к размеру исходного изображения. Объединённая маска определяется как поэлементный максимум:

$$M = \max(m_1, m_2, \dots, m_k).$$

Цветовое представление маски формируется путём назначения каждому типу повреждения собственного цвета. Тогда композитное изображение можно представить как:

$$X^{\text{comp}} = (1 - \alpha M) \odot X + \alpha M \odot C,$$

где  $\alpha$  - коэффициент прозрачности,  $C$  - цветовая карта маски, а  $\odot$  - поэлементное умножение.

## 5.4 Математическая модель расчёта стоимости и длительности ремонтных работ

После завершения автоматического анализа изображения или ручного ввода повреждений требуется определить ориентировочную стоимость и длительность ремонтных работ.

Пусть для каждого найденного повреждения известны:

- Тип детали ( $p_i$ );
- Тип повреждения ( $d_i$ ).

Для каждого повреждения стоимость и длительность определяются как табличные отображения - каждая пара (тип детали, тип повреждения) однозначно соответствует фиксированным значениям стоимости и длительности работ.

Итоговая стоимость ремонта вычисляется как:

$$C = \sum_{i \in A} c_i,$$

а итоговая длительность ремонта:

$$T = \sum_{i \in A} t_i.$$

где  $A$  - множество найденных повреждений.

Таблицы 5 и 6, которые были описаны в главе 4, позволяют однозначно задать стоимость и длительность работ для каждой пары "деталь - тип повреждения". Это делает расчёт прозрачным и позволяет изменять ценники и правила автосервиса без изменения алгоритма системы.

## 6 Алгоритм решения задачи

На UML Activity диаграмме показан алгоритм решения задачи (рисунок 17). Система реализует единый сценарий оценки стоимости и длительности ремонтных работ с разветвлением на два режима - автоматический (с использованием моделей компьютерного зрения) и ручной (без использования ML).

На диаграмме представлены 4 "участника" процесса: Пользователь, Bot Service, Backend и ML Worker. Рассмотрим алгоритм взаимодействия пользователя и системы подробнее:

1. Пользователь инициирует взаимодействие с системой, отправляя команду /start или нажимая кнопку "Начать" в боте в мессенджере. Взаимодействие с сервером обеспечивает Bot Service, который передаёт команды из мессенджера системе. На сервере формируется приветственное сообщение и выводится пользователю в мессенджере. Оно содержит клавиатуру с возможностью выбрать режим работы - "С фотографией" и "Ручной ввод". Здесь пользователь выбирает режим, и сценарий разветвляется.
2. ML-режим: создание заявки. Сценарий переводится в статус CREATED, а Bot Service отображает пользователю сообщение об отправке фотографии повреждённого автомобиля.
3. После загрузки фотографии она передаётся серверу, валидируется и отправляется в объектное хранилище S3. Если формат был некорректен, пользователю отображается сообщение об ошибке формата и запрашивается повторная отправка изображения. Если в рамках сообщения приложено несколько фотографий (что позволяет сделать ВКонтакте или другой мессенджер), то пользователю отображается сообщение о том, что в рамках системы обрабатывается только первая фотография.
4. После валидации изображения статус переводится в QUEUED, и формируется задача инференса. Пользователь получает сообщение о начале обработки.
5. Далее в ML Worker выполняется предобработка и последующий анализ изображения. На первом этапе выполняется детекция и сегментация деталей автомобиля с помощью предобученной модели. По-



лученные результаты фильтруются по порогу уверенности, и, если деталей не обнаружено - формируется сообщение об ошибке инференса, происходит переход на обработку результатов.

6. Если детали были обнаружены первой моделью, далее выполняется обрезка этих деталей на фотографии и детекция и сегментация повреждений на каждой из обрезанных областей. После этого формируется аннотированное изображение, и происходит переход на обработку результатов. Если на этапе распознавания деталей или повреждений не получен пригодный результат, система переводит пользователя в сценарий ручного ввода, сохраняя возможность дальнейшего расчета стоимости.
7. При выборе пользователем ручного режима создаётся заявка со статусом PRICING, минуя этапы инференса с помощью ML-моделей.
8. Далее происходит переход к фазе, общей для обоих режимов - отображается клавиатура с доступными действиями по формированию / редактированию списка повреждений, в зависимости от выбора пользователь может добавить, удалить, отредактировать повреждение. Отображаются только уникальные сочетания вида <деталь-повреждение>, что упрощает восприятие и редактирование пользователем.
9. После окончания редактирования пользователь нажимает кнопку "Подтвердить и система переходит к расчёту стоимости на основании заданных параметров. Статус переводится из PRICING в DONE, а пользователю отображается финальное сообщение о стоимости и длительности работ.

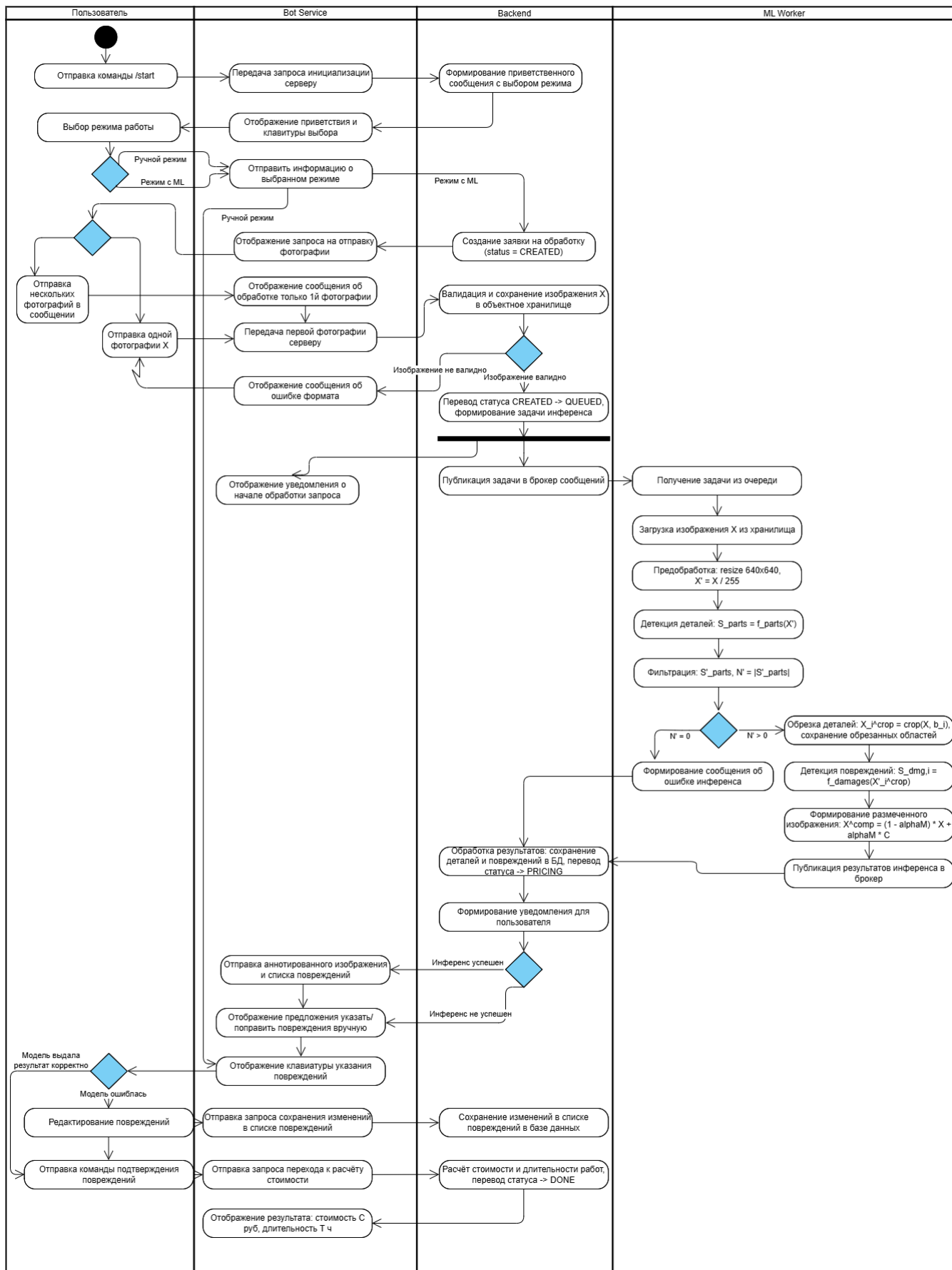


Рисунок 17 — UML Activity диаграмма сценария системы

## 7 Программная реализация

В данном разделе описана реализация проекта. Разработка ведётся на языке Python 3 в среде разработки Visual Studio Code для единой работы с репозиторием - все конфигурационные файлы, код и тесты располагаются в единой структуре проекта, что упрощает сопровождение. Компоненты системы упаковываются в Docker-контейнеры и оркестрируются с помощью Docker Compose (контейнеры: PostgreSQL, Kafka с Zookeeper, воркеры машинного обучения, бэкенд, сервис бота, nginx). Данный подход позволяет воспроизвести рабочую среду как локально на машине разработчика, так и на сервере развёртывания.

### 7.1 Программная реализация пайплайна обучения моделей и инференса

В рамках пайплайна анализа изображения для решения подзадач нахождения деталей и повреждений автомобиля на фотографии используется семейство моделей YOLOv8 в режиме segment через библиотеку Ultralytics. Базовые веса задаются предобученной моделью yolov8m-seg.pt, затем обучение выполняется на прикладных данных.

Стек технологий для обучения моделей и инференса:

- Python3 - реализация кода обучения и оценки;
- Ultralytics (YOLOv8) - обучение и инференс моделей сегментации;
- Numpy - быстрые численные операции в рамках приложения;
- Pillow, OpenCV (opencv-python-headless) - загрузка изображений, предобработка.

Для обучения модели обнаружения и сегментации деталей было размечено порядка 250 изображений. На рисунке 18 представлен пример разметки деталей автомобиля на изображении. Экспорт размеченных данных производится в формате Ultralytics YOLO Segmentation 1.0. На рисунке 19 показано, как выглядят размеченные данные, поступающие на вход на обучение модели.

Каждая строка размеченных данных соответствует одному объекту и содержит идентификатор класса и последовательность нормализованных координат (x, y), описывающих контур объекта (маску). Координаты заданы в



Рисунок 18 — Пример разметки изображения для модели обнаружения деталей

```
test > labels > Train > 105.txt
1 10 0.212191 0.873457 0.174383 0.874486 0.150463 0.875514 0.142747 0.877572 0.132716 0.
2 4 0.000000 0.000000 0.000000 0.273663 0.079475 0.272634 0.084105 0.272634 0.094136 0.
3 1 0.013117 0.280864 0.000000 0.281893 0.000000 0.895062 0.004630 0.895062 0.014660 0.
4 11 0.939815 0.581276 0.938272 0.583333 0.938272 0.587449 0.936728 0.590535 0.935185 0.
5 6 0.844907 0.355967 0.844136 0.358025 0.844136 0.360082 0.841821 0.363169 0.841821 0.
6
```

Рисунок 19 — Пример размеченных данных

относительной системе (в диапазоне  $[0, 1]$ ) относительно ширины и высоты изображения.

Аналогичная разметка была проведена для обучения модели сегментации повреждений. Для данной модели было размечено ориентировочно 370 изображений. Пример размеченного обрезанного изображения детали представлен на рисунке 20.

## 7.2 Программная реализация серверной части

Серверная реализация строится на многослойной архитектуре и ориентирована на асинхронное выполнение. Стек бэкенда:

- FastAPI - фреймворк для REST API;
- uvicorn - ASGI-сервер для запуска приложения;

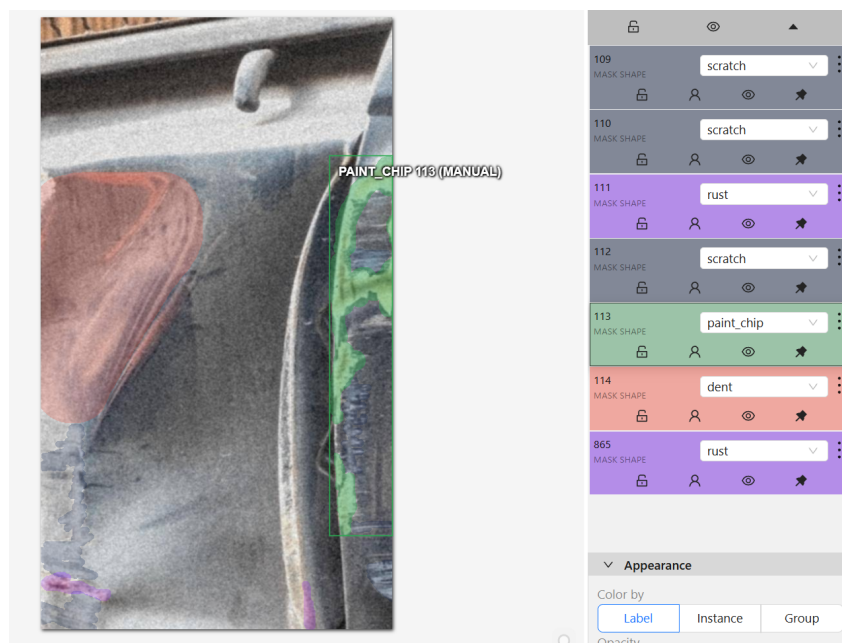


Рисунок 20 — Пример разметки изображения для модели обнаружения повреждений

- asyncio - библиотека для асинхронного выполнения кода;
- asyncpg - для асинхронного доступа к PostgreSQL;
- aiokafka - асинхронный клиент Apache Kafka;
- MinIO (S3 API) - хранение исходных и аннотированных изображений;
- Pydantic - удобная валидация данных и конфигурация (pydantic-settings);
- Loguru - структурированное логирование в сервисах.

Реализованы и используются механизмы обеспечения отказоустойчивости и согласованности данных:

- Обмен данными между бэкендом и ML-воркерами производится через топики Kafka;
- Паттерн Transactional Outbox: фиксируются события и доставка в брокер сообщений, снимает риск рассогласования вида "запись в БД прошла, а сообщение в Kafka не дошло";
- Реализованы таймауты и отслеживание статусов каждой заявки.

### 7.3 Реализация интерфейса в мессенджере

Сценарий для пользователя реализуется в боте Вконтакте с использованием фреймворка vkbot - асинхронный long poll, обработка сообщений и

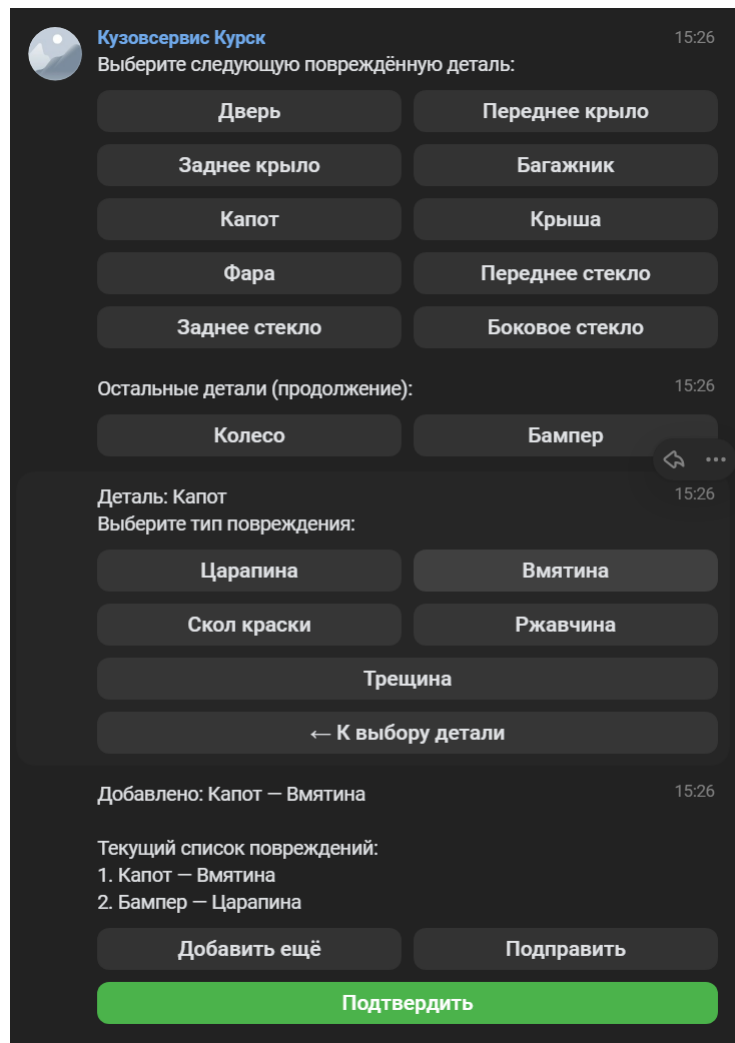


Рисунок 21 — Интерфейс бота Вконтакте

callback кнопок клавиатуры. Бот не обращается к ML напрямую, а вызывает HTTP API бэкенда и реагирует на уведомления от Kafka.

В ручном режиме ("Ручной ввод") пользователь последовательно выбирает деталь и тип повреждения, а в ML-режиме - отправляет фотографию и производится автоматический анализ. В рамках реализации учтены ограничения платформы Вконтакте на размер и состав inline-клавиатур - длинное меню при выборе деталей разбито на несколько сообщений.

## 7.4 Тестирование

Функциональное и юнит-тестирование реализовано с помощью pytest и pytest-asyncio - проверяются единицы поведения доменных сервисов, сценарии бэкенда, клиент бота, генерация клавиатуры и пр. Для проверки работы кода, связанного с контроллерами (БД), используются интеграцион-

ные тесты с использованием библиотеки testcontainers, позволяющей поднять PostgreSQL в контейнере во время работы теста.

Реализована и внедрена непрерывная интеграция (CI/CD) через Github Actions - при каждом залитии новых изменений в ветку код проверяется на стиль и статический анализ (ruff, flake8 и mypy), собирается окружение и прогоняются все тесты проекта с порогом покрытия 70%.

Также было проведено нагрузочное тестирование - написан скрипт на Python3, который отправляет более 100 запросов в секунду, содержащих изображения, для проверки отказоустойчивости системы. Фиксируются кол-во успешно обработанных сценариев, время отклика, а также другие детали работы системы, такие как, например, заполнение очереди в брокере сообщений.

## 8 Анализ результатов

### 8.1 Результаты обучения моделей компьютерного зрения

Для решения задачи автоматизированного анализа фотографии на предмет повреждений автомобиля был реализован двухэтапный ML-конвейер. Подход разделения задачи на два этапа - поиска деталей и поиска повреждений - позволил устранить проблему логически некорректных сочетаний (например, <фара - спущенная шина>), а также приблизить модель к реальному сценарию работы автосервиса, где модель, как и человек, анализирует повреждения в рамках отдельных деталей автомобиля.

В ходе работы были обучены две модели компьютерного зрения на базе YOLOv8-seg:

- Модель сегментации деталей автомобиля;
- Модель сегментации повреждений автомобиля.

Таблица 7 — Итоговые метрики модели сегментации деталей

| Метрика                      | Значение        |
|------------------------------|-----------------|
| Размер обучающей выборки     | 174 изображения |
| Размер валидационной выборки | 26 изображений  |
| Размер тестовой выборки      | 26 изображений  |
| Precision (Box)              | 0.766           |
| Recall (Box)                 | 0.517           |
| mAP@0.5 (Box)                | 0.560           |
| mAP@0.5:0.95 (Box)           | 0.471           |
| Начальный box_loss           | 1.171           |
| Конечный box_loss            | 0.3387          |
| Количество итераций (эпох)   | 150             |
| % падения loss               | 346%            |



Таблица 8 — Итоговые метрики модели сегментации повреждений

| Метрика                      | Значение                    |
|------------------------------|-----------------------------|
| Размер обучающей выборки     | 313 изображений             |
| Размер валидационной выборки | 29 изображений              |
| Размер тестовой выборки      | 30 изображений              |
| Precision (Mask)             | 0.684                       |
| Recall (Mask)                | 0.449                       |
| mAP@0.5 (Mask)               | 0.474                       |
| mAP@0.5:0.95 (Mask)          | 0.307                       |
| Начальный seg_loss           | 3.463                       |
| Конечный seg_loss            | 1.377                       |
| Количество итераций          | 137 (137/150 EarlyStopping) |
| % падения loss               | 251%                        |

Результаты обучения обеих моделей визуализированы и представлены на графиках. На рисунках 22 и 23 представлены графики loss и метрик, а на рисунках 24 и 25 - скриншоты процессов обучения моделей.

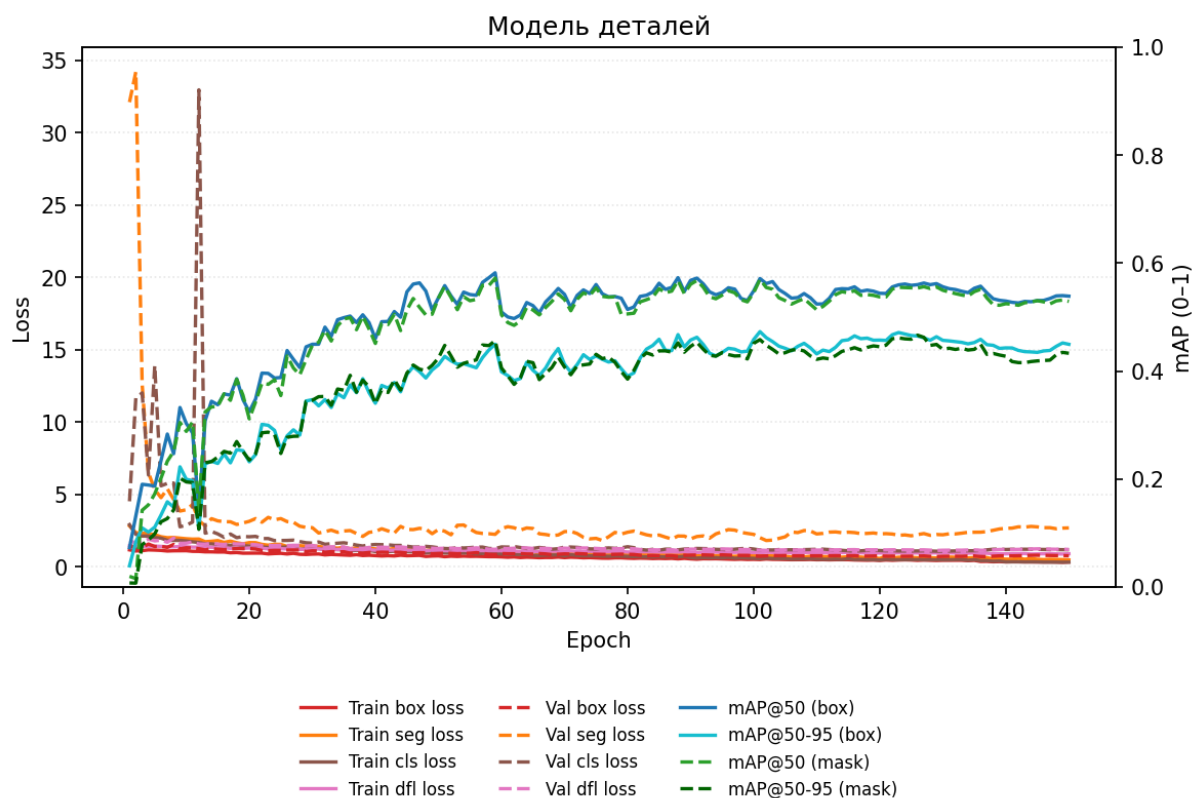


Рисунок 22 — График метрик обучения модели деталей

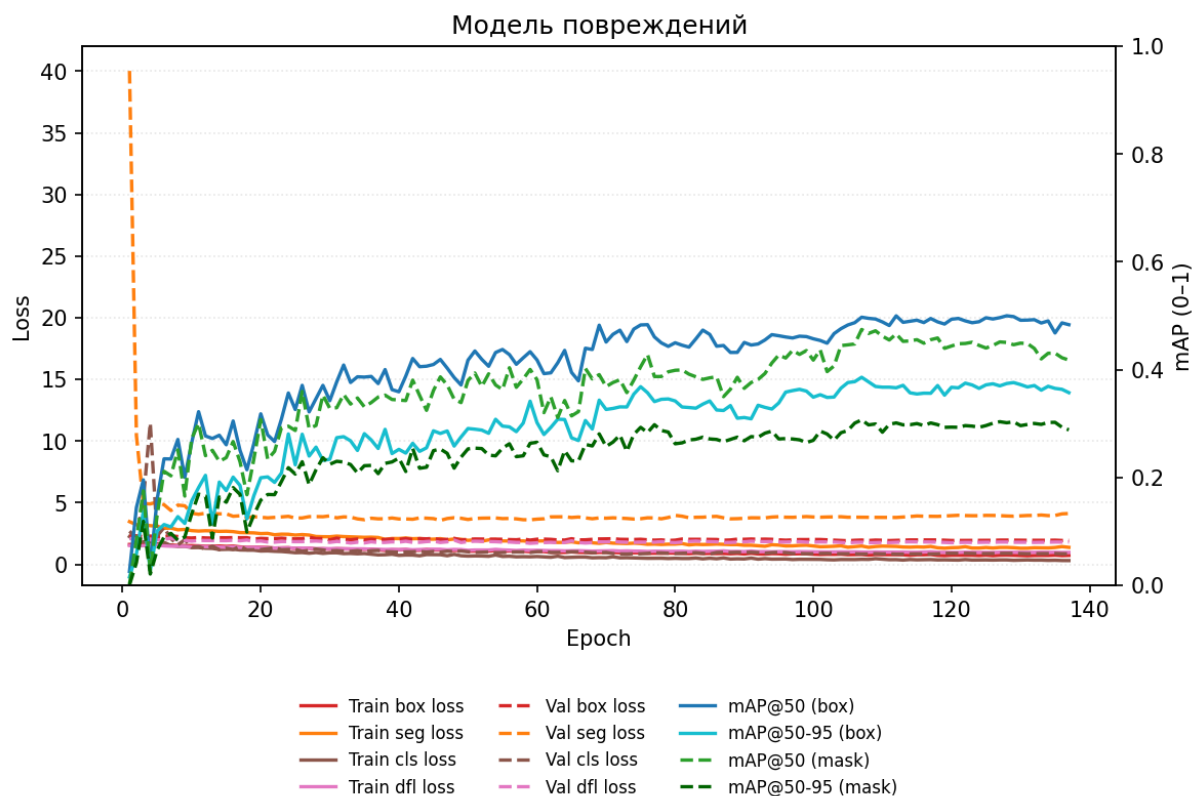


Рисунок 23 — График метрик обучения модели повреждений

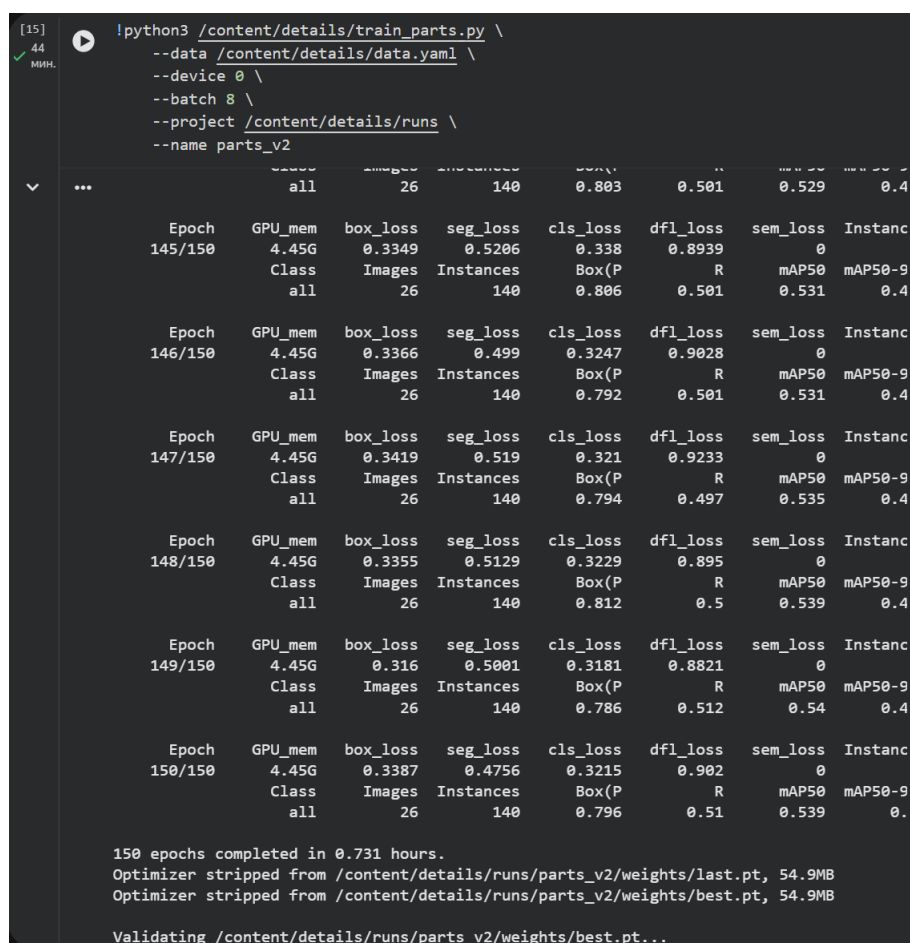


Рисунок 24 — Скриншот процесса обучения модели деталей

```

[3] python3 /content/damages/train_damages.py \
--data /content/damages/data.yaml \
--device 0 \
--batch 8 \
--project /content/runs \
--name damages_v2

```

|                                                                                                                                                                           |         | all      | Images    | Instances | Box(P    | R        | mAP50 | mAP |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------|----------|-----------|-----------|----------|----------|-------|-----|
| Epoch                                                                                                                                                                     | GPU_mem | box_loss | seg_loss  | cls_loss  | df1_loss | sem_loss | In:   |     |
| 132/150                                                                                                                                                                   | 4.66G   | 0.7182   | 1.346     | 0.3412    | 0.9737   | 0        |       |     |
|                                                                                                                                                                           | Class   | Images   | Instances | Box(P     | R        | mAP50    | mAP   |     |
|                                                                                                                                                                           | all     | 29       | 186       | 0.59      | 0.469    | 0.493    |       |     |
| Epoch                                                                                                                                                                     | GPU_mem | box_loss | seg_loss  | cls_loss  | df1_loss | sem_loss | In:   |     |
| 133/150                                                                                                                                                                   | 4.66G   | 0.7249   | 1.338     | 0.344     | 0.9857   | 0        |       |     |
|                                                                                                                                                                           | Class   | Images   | Instances | Box(P     | R        | mAP50    | mAP   |     |
|                                                                                                                                                                           | all     | 29       | 186       | 0.565     | 0.487    | 0.486    |       |     |
| Epoch                                                                                                                                                                     | GPU_mem | box_loss | seg_loss  | cls_loss  | df1_loss | sem_loss | In:   |     |
| 134/150                                                                                                                                                                   | 4.66G   | 0.7178   | 1.331     | 0.322     | 0.9757   | 0        |       |     |
|                                                                                                                                                                           | Class   | Images   | Instances | Box(P     | R        | mAP50    | mAP   |     |
|                                                                                                                                                                           | all     | 29       | 186       | 0.638     | 0.457    | 0.49     |       |     |
| Epoch                                                                                                                                                                     | GPU_mem | box_loss | seg_loss  | cls_loss  | df1_loss | sem_loss | In:   |     |
| 135/150                                                                                                                                                                   | 4.66G   | 0.744    | 1.367     | 0.3327    | 0.9762   | 0        |       |     |
|                                                                                                                                                                           | Class   | Images   | Instances | Box(P     | R        | mAP50    | mAP   |     |
|                                                                                                                                                                           | all     | 29       | 186       | 0.617     | 0.422    | 0.468    |       |     |
| Closing dataloader mosaic                                                                                                                                                 |         |          |           |           |          |          |       |     |
| albumentations: Blur(p=0.01, blur_limit=(3, 7)), MedianBlur(p=0.01, blur_limit=(3, 7))                                                                                    |         |          |           |           |          |          |       |     |
| Epoch                                                                                                                                                                     | GPU_mem | box_loss | seg_loss  | cls_loss  | df1_loss | sem_loss | In:   |     |
| 136/150                                                                                                                                                                   | 4.66G   | 0.7077   | 1.44      | 0.3098    | 0.9633   | 0        |       |     |
|                                                                                                                                                                           | Class   | Images   | Instances | Box(P     | R        | mAP50    | mAP   |     |
|                                                                                                                                                                           | all     | 29       | 186       | 0.554     | 0.489    | 0.487    |       |     |
| Epoch                                                                                                                                                                     | GPU_mem | box_loss | seg_loss  | cls_loss  | df1_loss | sem_loss | In:   |     |
| 137/150                                                                                                                                                                   | 4.66G   | 0.7247   | 1.377     | 0.3074    | 0.9683   | 0        |       |     |
|                                                                                                                                                                           | Class   | Images   | Instances | Box(P     | R        | mAP50    | mAP   |     |
|                                                                                                                                                                           | all     | 29       | 186       | 0.54      | 0.471    | 0.483    |       |     |
| EarlyStopping: Training stopped early as no improvement observed in last 30 epoch:<br>To update EarlyStopping(patience=30) pass a new patience value, i.e. `patience=300` |         |          |           |           |          |          |       |     |
| 137 epochs completed in 0.754 hours.                                                                                                                                      |         |          |           |           |          |          |       |     |

Рисунок 25 — Скриншот процесса обучения модели повреждений

По полученным графикам можно сделать вывод о том, что модели демонстрируют сходимость в ходе обучения (снижение loss - усвоение закономерностей). Не менее показательными при этом являются не только значения функции потерь, но и итоговые значения precision, recall и mAP на валидационной и тестовой выборках (таблицы 7 и 8) - именно они показывают пригодность моделей к задаче.

Если сравнивать результаты двух моделей, то модель сегментации деталей демонстрирует результаты лучше, чем модель сегментации повреждений. Это объясняется рядом причин:

- Автомобильные детали намного крупнее и имеют в большинстве случаев схожую форму;
- Повреждения более вариативны по форме, размеру, степени выраженности и условиям съёмки.

По этой причине задача сегментации повреждений является намного более сложной, а метрики по ней закономерно ниже.

С практической точки зрения сервис сохраняет прикладную ценность даже в случае неполного или неидеального распознавания - пользователь имеет возможность отредактировать результат и получить расчёт стоимости. Таким образом, ML-модели позволяют сократить объём ручной работы, но не выступают как полностью необратимая система принятия решений.

Важным результатом является визуальная интерпретируемость работы моделей. В конце работы пользователю возвращается аннотированное изображение с наложенными масками повреждений (рисунок 28), что позволяет оценить корректность работы модели. Это повышает доверие к сервису и облегчает проверку результата человеком.

На рисунках 26-30 представлены результаты работы моделей деталей и повреждений - аннотированные изображения.

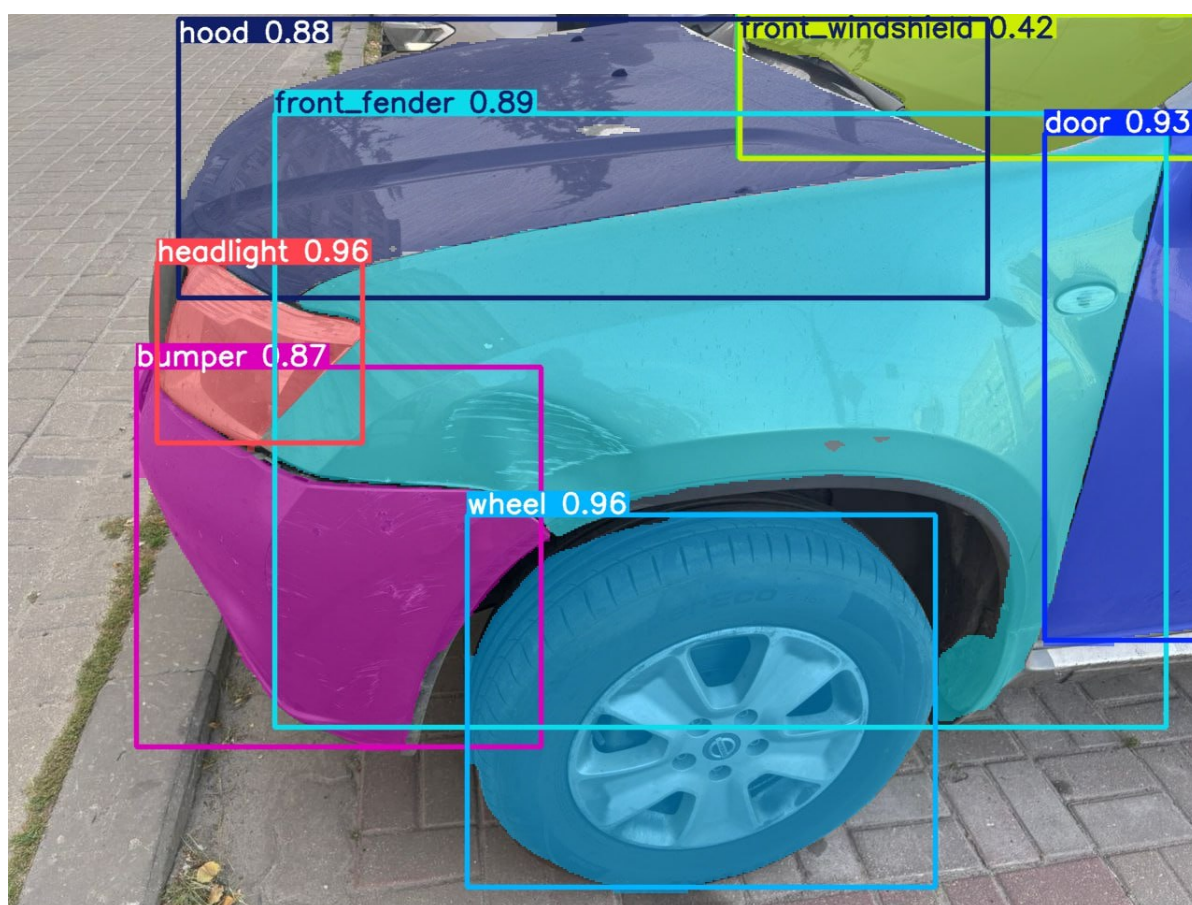


Рисунок 26 — Пример сегментации деталей на исходной фотографии



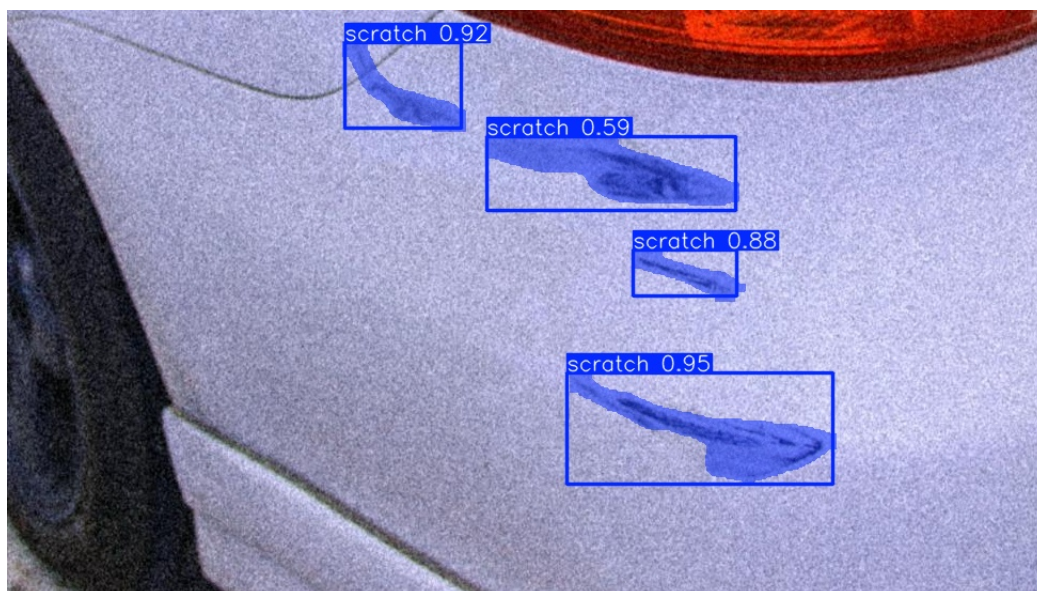


Рисунок 27 — Пример сегментации повреждений на обрезанных фрагментах деталей



Рисунок 28 — Пример итогового аннотированного изображения, отправляемого пользователю



Рисунок 29 — Пример корректного определения нескольких типов повреждений

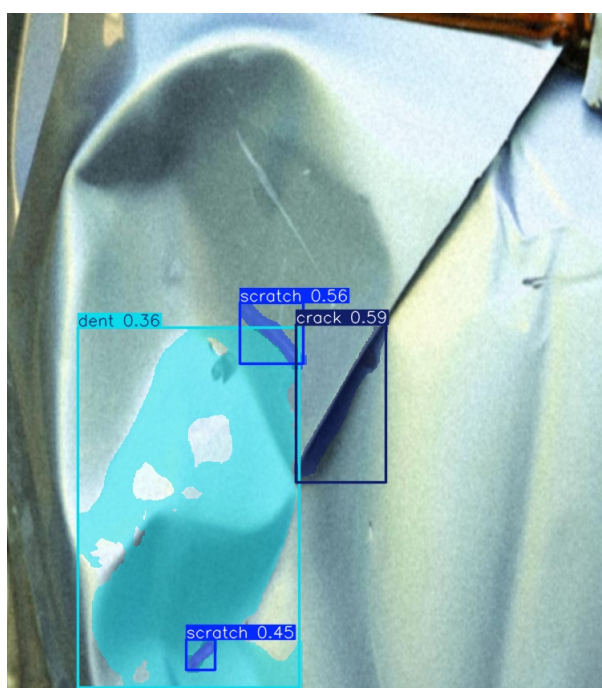


Рисунок 30 — Пример частично ошибочного определения, демонстрирующий ограничения модели

Анализируя полученные результаты, можно выделить ряд причин, по которым результат некоторых предсказаний может не соответствовать реальным повреждениям:

1. Ограничен объём датасета и размеченных фотографий

2. Дисбаланс классов - есть типы повреждений, которые встречаются намного реже частых, таких как царапины
3. Сложность самих визуальных объектов, в особенности мелких царапин или вмятин
4. Неоднозначность границ повреждений при разметке

Достигнутый результат по ML-части можно оценить как достаточный для задачи предварительной автоматизированной оценки, но требует дальнейшего улучшения за счет увеличения кол-ва различных размеченных фотографий, в особенности, повреждений.

## 8.2 Результаты нагрузочного тестирования

Таблица 9 — Результаты нагрузочного тестирования

| Целевая нагрузка | Успешность | Наблюдаемый RPS | p50 полного цикла | p95 полного цикла | Ошибки         |
|------------------|------------|-----------------|-------------------|-------------------|----------------|
| 50 RPS           | 100%       | 37.04 RPS       | 1.58 с            | 2.12 с            | -              |
| 120 RPS          | 100%       | 33.28 RPS       | 4.40 с            | 5.62 с            | -              |
| 200 RPS          | 93.34%     | 21.77 RPS       | 12.32 с           | 14.93 с           | 130x ReadError |

Нагрузочное тестирование проводилось для полного ML-сценария: создание заявки, загрузка изображения в S3-хранилище и постановка задачи инференса в Kafka. Его результаты представлены в таблице 9. При нагрузках 50 и 120 RPS обработка всех запросов прошла без ошибок. Это подтверждает устойчивость системы при повышенной нагрузке: все микросервисы работали штатно на протяжении тестирования.

На текущем одноузловом стенде система упирается примерно в 33–37 RPS. При попытке подать нагрузку выше этого значения система не выдаёт ошибок, но растёт задержка: p95 увеличивается с 2.12 с до 5.62 с.

При 200 RPS начинается деградация: успешность запросов снижается до 93.34%, а p95 достигает почти 15 секунд. Это показывает практический предел текущего локального стенда. Основная причина ограничения - запуск backend в одном uvicorn-процессе и размещение всех сервисов на одной машине.

Итог: система выдерживает стабильную обработку около 35 RPS без ошибок, корректно буферизует ML-задачи через Kafka и показывает устойчивость при умеренной перегрузке. Для достижения уровня выше 100 RPS

в продуктиве требуется горизонтальное масштабирование backend-сервиса и ML worker'ов.

### 8.3 Результаты работы сервиса

Пользователь взаимодействует с сервисом через бота в мессенджере Вконтакте. По итогам реализации сервис поддерживает 2 режима работы - автоматический с анализом фотографии и ручной режим ввода повреждений.

Пользовательский сценарий начинается с команды /start (или нажатия кнопки "Начать") с последующим выбором режима работы (Рисунок 31). При выборе режима с фотографией пользователь отправляет изображение автомобиля с видимым повреждением (Рисунок 32).

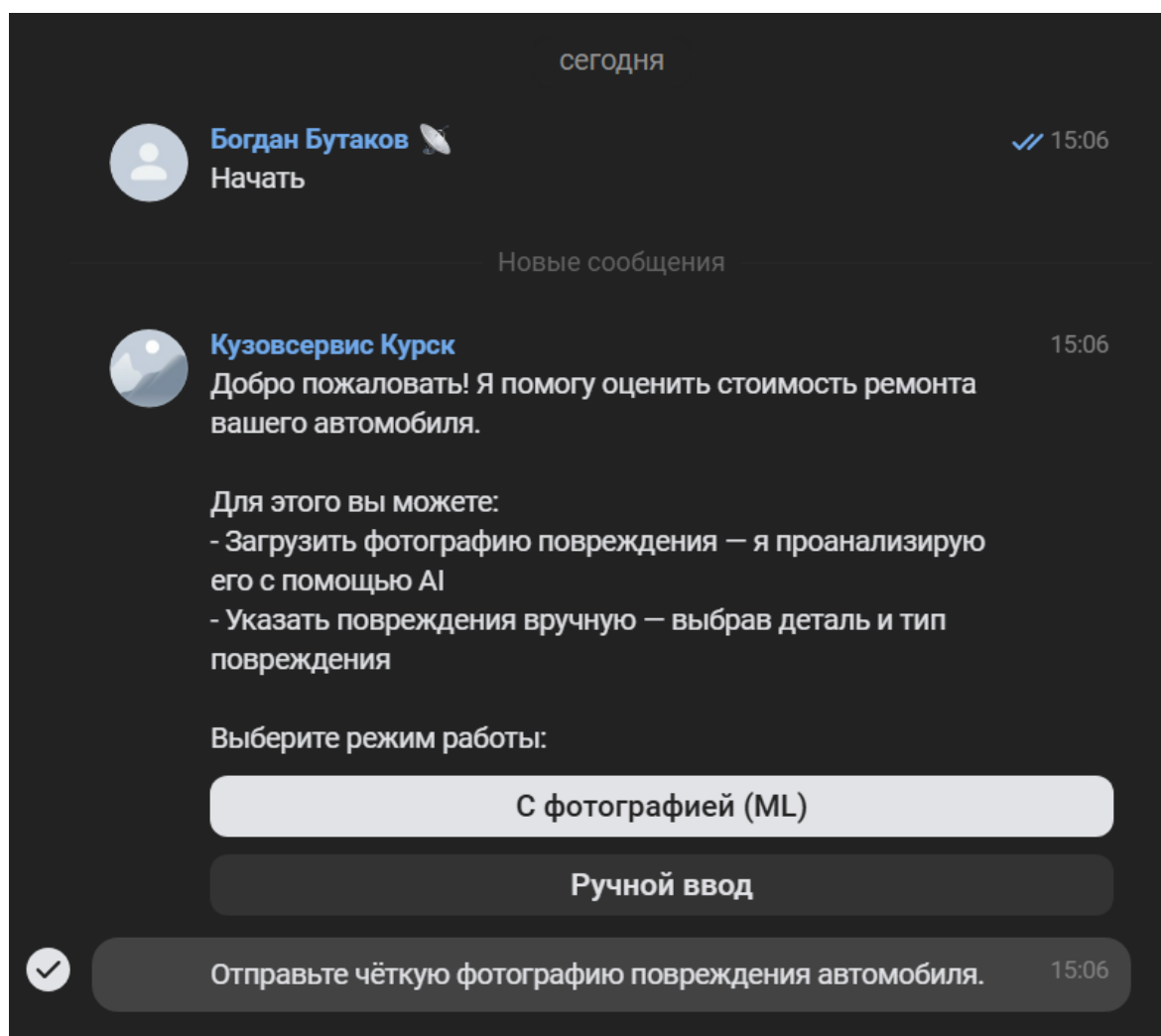


Рисунок 31 — Интерфейс бота (начальный этап выбора режима работы)



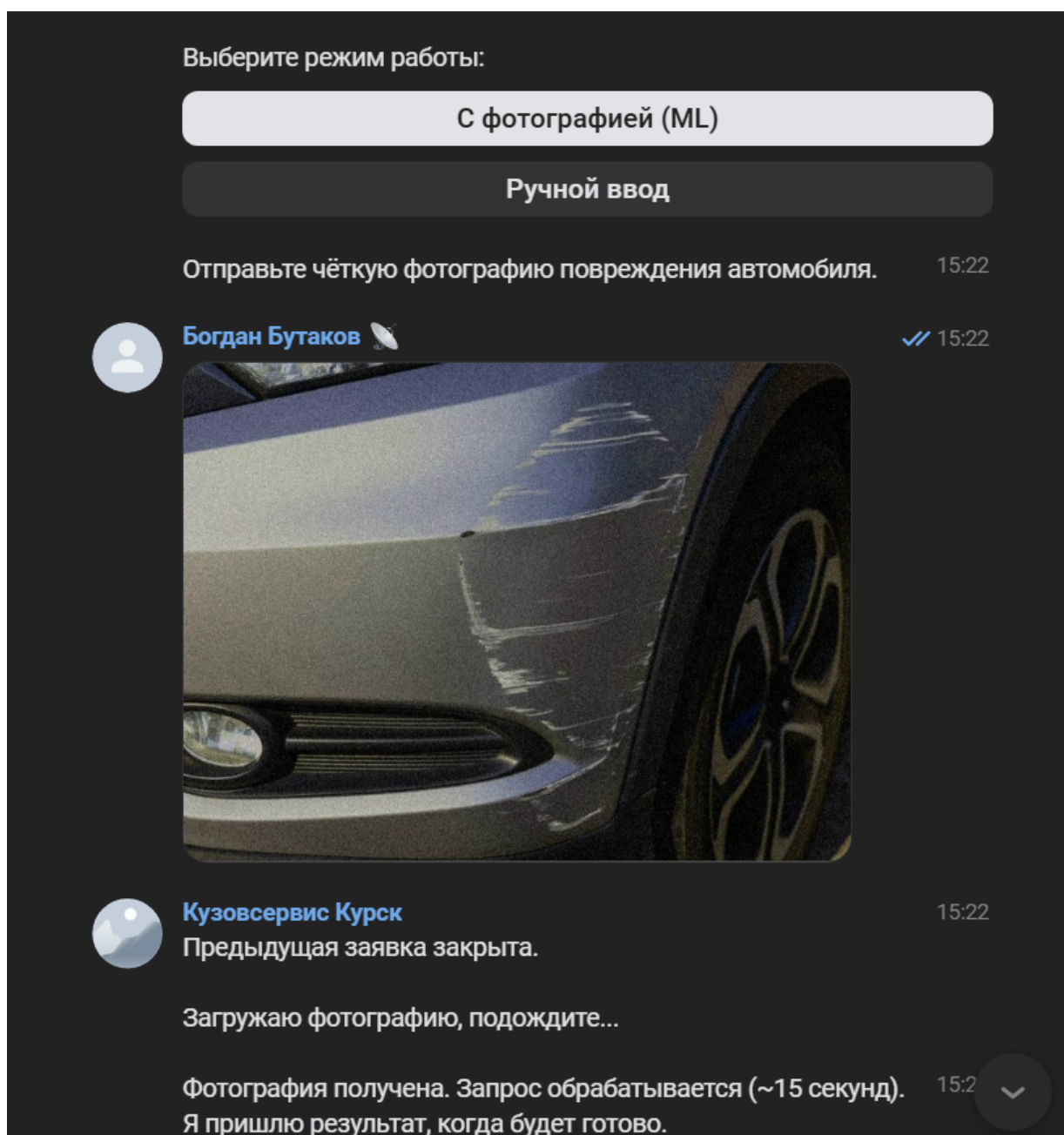


Рисунок 32 — Пример взаимодействия с ботом в режиме ML

Сервис сохраняет фотографию, после чего система выполняет обработку изображения и возвращает аннотированное изображение вместе со списком найденных повреждений и клавиатуру с возможностью отредактировать повреждения и кнопкой подтверждения (Рисунок 33).

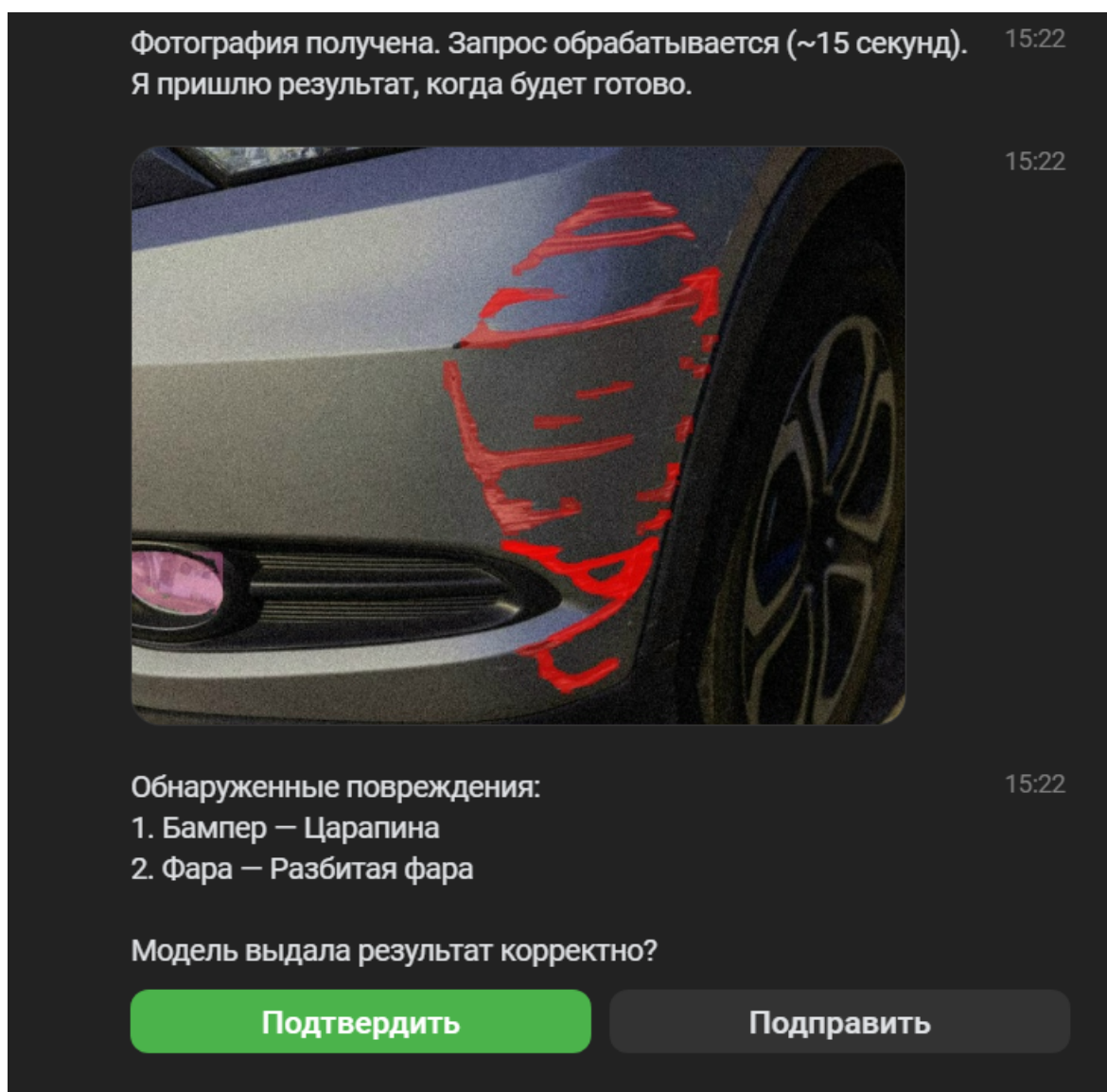


Рисунок 33 — Результат обработки фотографии в режиме ML

Существенным преимуществом в интерфейсе бота является функционал, где при необходимости пользователь может отредактировать список обнаруженных повреждений - удалить, изменить тип найденного повреждения или его принадлежность к детали, или добавить необнаруженное повреждение (Рисунки 34 и 35). Это делает систему более пригодной для практического пользования и не делает пользователя зависимым от возможных просчётов модели.



Кузовсервис Курск

15:25

Редактирование повреждений:

1. Фара — Разбитая фара

2. Бампер — Царапина

Фара — Разбитая фара

Бампер — Царапина

Добавить повреждение

Готово

Текущие повреждения:

15:25

1. Фара — Разбитая фара

2. Бампер — Царапина

Выберите новый тип повреждения:

Разбитая фара

Удалить повреждение

← К списку повреждений

Повреждение удалено.

15:25

Текущий список:

1. Бампер — Царапина

Бампер — Царапина

Добавить повреждение

Готово

Рисунок 34 — Редактирование списка повреждений (удаление)



Кузовсервис Курск

15:26

Выберите следующую повреждённую деталь:

Дверь

Переднее крыло

Заднее крыло

Багажник

Капот

Крыша

Фара

Переднее стекло

Заднее стекло

Боковое стекло

Остальные детали (продолжение):

15:26

Колесо

Бампер



Деталь: Капот

15:26

Выберите тип повреждения:

Царапина

Вмятина

Скол краски

Ржавчина

Трещина

← К выбору детали

Добавлено: Капот — Вмятина

15:26

Текущий список повреждений:

1. Капот — Вмятина

2. Бампер — Царапина

Добавить ещё

Подправить

Подтвердить

Рисунок 35 — Редактирование списка повреждений (добавление)

После того, как пользователь введёт полный список повреждений, он может нажать кнопку "Подтвердить и сервис сразу выдаст полную стоимость и длительность работ в рабочих часах (Рисунок 36):

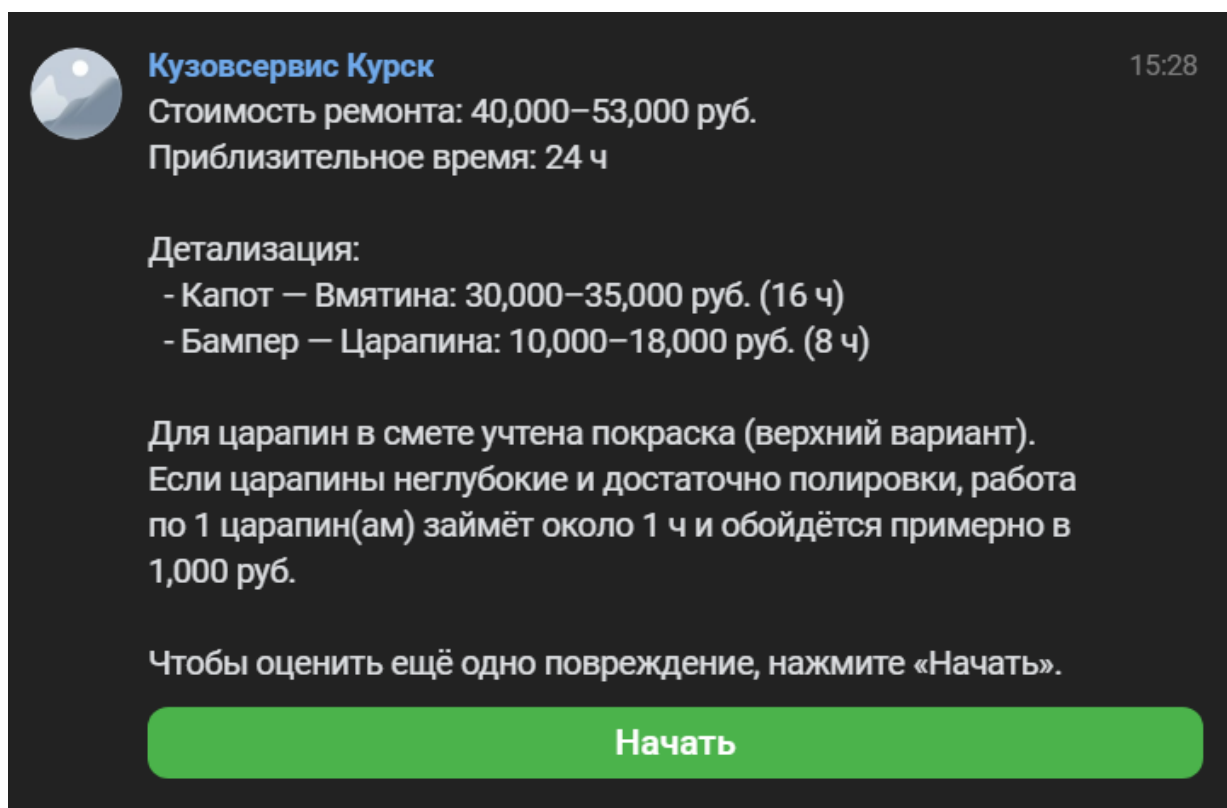


Рисунок 36 — Результат подсчёта стоимости и длительности работ

Пользователь может выбрать и режим ручного ввода повреждений. В этом режиме ему доступен схожий функционал, что и в режиме ML при редактировании повреждений, только теперь пользователь вводит все повреждения с нуля самостоятельно. (Рисунок 37)

×



Кузовсервис Курск

2 подписчика

🔍

⋮

Выберите режим работы:

С фотографией (ML)

Ручной ввод

Выберите повреждённую деталь:

15:39

Дверь

Переднее крыло

Заднее крыло

Багажник

Капот

Крыша

Фара

Переднее стекло

Заднее стекло

Боковое стекло

Остальные детали (продолжение):

15:39

Колесо

Бампер

Деталь: Переднее крыло

15:40

Выберите тип повреждения:

Царапина

Вмятина

Скол краски

Ржавчина

Трещина

← К выбору детали

Добавлено: Переднее крыло — Царапина

15:40

Текущий список повреждений:

1. Переднее крыло — Царапина

Добавить ещё

Подправить

Подтвердить

Рисунок 37 — Пример взаимодействия с ботом в ручном режиме

66

Таким образом, с точки зрения пользовательского опыта достигнут результат, при котором:

- Сценарий запуска является быстрым и понятным;
- Доступна свобода действий и выбора автоматического и ручного режима;
- Пользователь имеет удобный, не перегруженный интерфейс и может контролировать результат;
- Итоговая смета представляется в понятном виде.

#### **8.4 Оценка выполнения задач исследования**

В рамках работы был подготовлен датасет изображений деталей и повреждений и выполнена разметка в формате, пригодном для дообучения моделей YOLOv8-seg. Уровень решения задачи можно оценить как высокий с точки зрения работоспособного датасета и средний с точки зрения охвата всех редких случаев. Ограничением всё так же остаётся необходимости дальнейшего расширения и балансировки выборки, особенно для сложных и трудноразличимых повреждений.

Разработка модулей для обработки изображений также решена. На этапе обучения используется конфигурируемый набор аугментаций, что позволило модели быть устойчивой к разнообразию фотографий. Использование аугментаций также помогло частично компенсировать ограниченный объём размеченных данных. Недостатки съёмки, такие как нечёткие фотографии, блики, плохое освещение, всё же существенно снижают качество распознавания, так как учесть все возможные вариации съёмки и передать эту информацию модели невозможно. Однако обучение моделей показало, что они демонстрируют сходимость в ходе обучения, что видно по функции потерь и метрикам - это означает, что модели пригодны к задаче, и главной проблемой является необходимость большого количества грамотно размеченных данных.

Бизнес-логика оценки стоимости и длительности ремонта реализована в полном объёме. Система корректно учитывает все особенности предметной области и обладает знанием о всех расценках автосервиса и корректно применяет их ко всем парам деталей и повреждений, как это делают сотрудники автосервиса.

Система построена в виде микросервисной архитектуры, включающей Backend API, Bot Service и ML Worker. Обмен сообщениями осуществляется через Kafka, для хранения изображений используется S3, а данные заявок и повреждений сохраняются в PostgreSQL. Такое архитектурное решение позволило разделить ответственность между разными компонентами системы, сделать её масштабируемой. Интеграция с ботом в мессенджере выполнена - пользователь может полностью пройти весь сценарий оценки, не покидая интерфейс Вконтакте. Архитектура базы данных соответствует микросервисному подходу и поддерживает основные сценарии работы системы.

Также произведена интеграция CI/CD - настроен конвейер с использованием GitHub Actions, выполняющий автоматическую проверку стиля кода и запуск тестов при изменениях в целевой ветви. Система контейнеризирована с использованием Docker Compose, что обеспечивает воспроизводимую локальную сборку, и развёрнута на сервере.

Проведено полноценное тестирование работоспособности системы - в проекте реализовано более 500 юнит- и интеграционных тестов, которые поддерживают устойчивость реализованных сценариев на уровне кода. Дополнительно реализован отдельный скрипт нагрузочного тестирования, имитирующий полный сценарий ML режима работы: создание заявки, загрузка изображений в объектное хранилище. Это позволило оценить пропускную способность системы и устойчивость к интенсивному потоку запросов. Результаты нагрузочного тестирования были представлены выше в таблице 9.

На основании приведённого анализа можно сделать вывод о том, что все поставленные задачи получили практическое решение. Наибольший потенциал для дальнейшего улучшения остаётся в повышении качества ML-моделей и разметки большего количества фотографий деталей и повреждений.

Следует отметить, что достигнутый результат не означает полной замены эксперта-оценщика. Итоговая система является инструментом предварительной автоматизированной оценки, а не окончательным юридически значимым подсчётом стоимости ремонта.



## ЗАКЛЮЧЕНИЕ

Данная работа позволила достигнуть поставленной цели - повышения эффективности процесса оценки стоимости ремонта поврежденных автомобилей. Достижение данной цели оценивалось по двум основным факторам:

1. Производительность системы
2. Минимизация человеческого участия

Таблица 10 — Сравнение состояния до и после внедрения системы

| Критерий                                                      | Исходное состояние     | Результат после внедрения                          | Вывод                   |
|---------------------------------------------------------------|------------------------|----------------------------------------------------|-------------------------|
| Скорость получения предварительной сметы и длительности работ | 30 минут               | 1 минута                                           | Значительно сократилась |
| Возможность параллельной обработки обращений                  | Ограниченная           | Есть за счет микросервисной архитектуры и очередей | Цель достигнута         |
| Надежность и воспроизводимость сценария                       | Зависела от сотрудника | Формализована в виде системы и тестов              | Цель достигнута         |

С точки зрения производительности разработанный сервис обеспечивает автоматизированное прохождение сценария обработки обращения. Полученные в нагрузочном тестировании значения по пропускной способности и времени отклика показывают, что система способна выдержать поток обращений на уровне не менее 120 RPS и оставаться отказоустойчивой. Следовательно, первый критерий достижения цели можно считать выполненным.

С точки зрения минимизации человеческого участия цель также достигнута. В разработанном сервисе пользователь получает предварительный результат автоматически, и сотруднику автосервиса не требуется тратить время и отвлекаться на согласования и переписку с клиентом. Скорость получения ответа на интересующие вопросы клиента сокращена с 30 минут до 1 минуты, что высвобождает время сотрудников. Это позволит существенно оптимизировать текущий бизнес-процесс.

Степень достижения цели можно оценить как высокую, но не предельную. Разработанная система имеет перспективы дальнейшего развития:

1. Качество распознавания зависит от объёма и качества размеченных данных. Необходимо расширение выборки, особенно по редким и сложным классам.
2. Текущая версия системы анализирует одно изображение в рамках одного сценария, тогда как в реальной практике для более точной оценки может потребоваться несколько фотографий с разных ракурсов. Развитие системы в сторону анализа нескольких фотографий позволит избежать такого рода проблем.
3. В текущей версии системы проводится классификация повреждений без оценки тяжести дефектов. Например, оценка глубины или площади вмятин позволила бы точнее рассчитывать стоимость ремонта.

## СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Automated Car Damage Assessment Using Computer Vision: Insurance Company Use Case. — 2024. — URL: <https://www.mdpi.com/2076-3417/14/20/9560> ; [Online]. Applied Sciences (MDPI), Vol. 14, No. 20.
2. Vehicle Damage Detection Using Artificial Intelligence: A Systematic Literature Review. — 2025. — URL: [https://www.researchgate.net/publication/392493928\\_Vehicle\\_Damage\\_Detection\\_Using\\_Artificial\\_Intelligence\\_A\\_Systematic\\_Literature\\_Review](https://www.researchgate.net/publication/392493928_Vehicle_Damage_Detection_Using_Artificial_Intelligence_A_Systematic_Literature_Review) ; [Online]. ResearchGate.
3. Convolutional Neural Networks for vehicle damage detection. — 2022. — URL: <https://www.sciencedirect.com/science/article/pii/S2666827022000433> ; [Online]. Results in Engineering.
4. HL-YOLO: Improving Vehicle Damage Detection with Heterogeneous Convolutions and Large-Kernel Attention. — 2025. — URL: <https://www.mdpi.com/2032-6653/16/12/640> ; [Online]. World Electric Vehicle Journal (MDPI), Vol. 16, No. 12.
5. Применение сверточных нейронных сетей Mask R-CNN в интеллектуальных парковочных системах. — 2020. — URL: <https://cyberleninka.ru/article/n/primenenie-svertochnyh-neyronnyh-setey-mask-r-cnn-v-intellektualnyh-parkovochnyh-sistemah/viewer> ; [Online]. Интернет-ресурс (CyberLeninka).
6. Smart Car Damage Assessment Using Enhanced YOLO Algorithm and Image Processing Techniques. — 2025. — URL: <https://www.mdpi.com/2078-2489/16/3/211> ; [Online]. Information (MDPI), Vol. 16, No. 3.
7. Vehicle damage severity estimation for insurance operations using in-the-wild mobile images. — 2024. — URL: <https://ieeexplore.ieee.org/document/10194904> ; [Online]. IEEE / QUB technical report.
8. Evaluation of deep learning algorithms for semantic segmentation of car parts. — 2021. — URL: [https://www.researchgate.net/publication/351787893\\_Evaluation\\_of\\_deep\\_learning\\_algorithms\\_for\\_semantic\\_segmentation\\_of\\_car\\_parts](https://www.researchgate.net/publication/351787893_Evaluation_of_deep_learning_algorithms_for_semantic_segmentation_of_car_parts) ; [Online]. ResearchGate / preprint.

9. Optimized car parts detection with advanced feature fusion and attention modules. — 2025. — URL: <https://www.nature.com/articles/s41598-025-29855-w> ; [Online]. Scientific Reports (Nature).
10. Automatic damaged vehicle estimator using enhanced deep learning algorithm. — 2023. — URL: <https://www.sciencedirect.com/science/article/pii/S2667305323000170> ; [Online]. Results in Engineering.
11. Small Dents, Big Impact: A Dataset and Deep Learning Approach for Vehicle Dent Detection. — 2025. — URL: <https://arxiv.org/abs/2508.15431> ; [Online]. arXiv preprint arXiv:2508.15431.
12. Ultralytics: Explore Ultralytics YOLOv8. — 2024. — URL: <https://docs.ultralytics.com/models/yolov8/> ; [Online]. Ultralytics Documentation.
13. A comprehensive review on YOLO versions for object detection. — 2025. — URL: <https://www.sciencedirect.com/science/article/pii/S2215098625002162> ; [Online]. Measurement (ScienceDirect).
14. A Deep Learning and Transfer Learning Approach for Vehicle Damage Detection. — 2021. — URL: [https://www.researchgate.net/publication/351571025\\_A\\_Deep\\_Learning\\_and\\_Transfer\\_Learning\\_Approach\\_for\\_Vehicle\\_Damage\\_Detection](https://www.researchgate.net/publication/351571025_A_Deep_Learning_and_Transfer_Learning_Approach_for_Vehicle_Damage_Detection) ; [Online]. ResearchGate / preprint.
15. Automotive Parts Assessment — Applying Real-time Instance-Segmentation Models to Identify Vehicle Parts. — 2022. — URL: [https://www.researchgate.net/publication/358307017\\_Automotive\\_Parts\\_Assessment\\_Applying\\_Real-time\\_Instance-Segmentation\\_Models\\_to\\_Identify\\_Vehicle\\_Parts](https://www.researchgate.net/publication/358307017_Automotive_Parts_Assessment_Applying_Real-time_Instance-Segmentation_Models_to_Identify_Vehicle_Parts) ; [Online]. ResearchGate / conference paper.
16. Image Annotation and Labeling. — 2023. — URL: [https://www.researchgate.net/publication/380695470\\_Image\\_Annotation\\_and\\_Labeling](https://www.researchgate.net/publication/380695470_Image_Annotation_and_Labeling) ; [Online]. ResearchGate / article.
17. Synthetic Data Generation for Training Car Damage Detection Models. — 2024. — URL: [https://www.researchgate.net/publication/390805655\\_Synthetic\\_Data\\_Generation\\_for\\_Training\\_Car\\_Damage\\_Detection\\_Models](https://www.researchgate.net/publication/390805655_Synthetic_Data_Generation_for_Training_Car_Damage_Detection_Models) ; [Online]. ResearchGate / preprint.
18. Ultralytics: Car Parts Segmentation Dataset and Training Guidelines. — 2024. — URL: <https://docs.ultralytics.com/ru/datasets/segment/carparts-seg/> ; [Online]. Ultralytics Documentation.

19. CrashCar101: Procedural Generation for Damage Assessment. — 2023. — URL: [https://www.researchgate.net/publication/379708541\\_CrashCar101\\_Procedural\\_Generation\\_for\\_Damage\\_Assessment](https://www.researchgate.net/publication/379708541_CrashCar101_Procedural_Generation_for_Damage_Assessment) ; [Online]. ResearchGate / preprint.
20. CarDD: A New Dataset for Vision-based Car Damage Detection. — 2022. — URL: <https://arxiv.org/abs/2211.00945> ; [Online]. arXiv preprint arXiv:2211.00945.
21. Learning Part Segmentation through Unsupervised Domain Adaptation from Synthetic Vehicles. — 2023. — URL: <https://ieeexplore.ieee.org/document/9878600> ; [Online]. IEEE Xplore / conference paper.
22. CVAT: Computer Vision Annotation Tool — Documentation. — 2024. — URL: <https://docs.cvat.ai/docs/> ; [Online]. Online documentation.

## Приложение А

### Исходный код и описание структуры репозитория

Исходный код разработанной системы размещён в открытом репозитории на платформе GitHub:

`https://github.com/ButakovBI/AutoRepairEstimator`

Система построена на микросервисной архитектуре. Основной код системы находится в директории `src/auto_repair_estimator/` и разделён на три сервиса:

- `backend/` - FastAPI-сервис: API, бизнес-логика, компоненты отказоустойчивости;
- `bot/` - VK-бот: интерфейс взаимодействия с пользователем;
- `ml_worker/` - ML-воркер: обработка изображений и инференс.

Прочие директории репозитория:

- `ml/` - скрипты и материалы для обучения моделей;
- `docker/` - файлы `Dockerfile` и `docker-compose.yml`;
- `tests/` - юнит- и интеграционные тесты;
- `scripts/` - вспомогательные скрипты, в том числе нагрузочное тестирование;
- `.github/workflows/` - конфигурация CI/CD пайплайна на GitHub Actions.